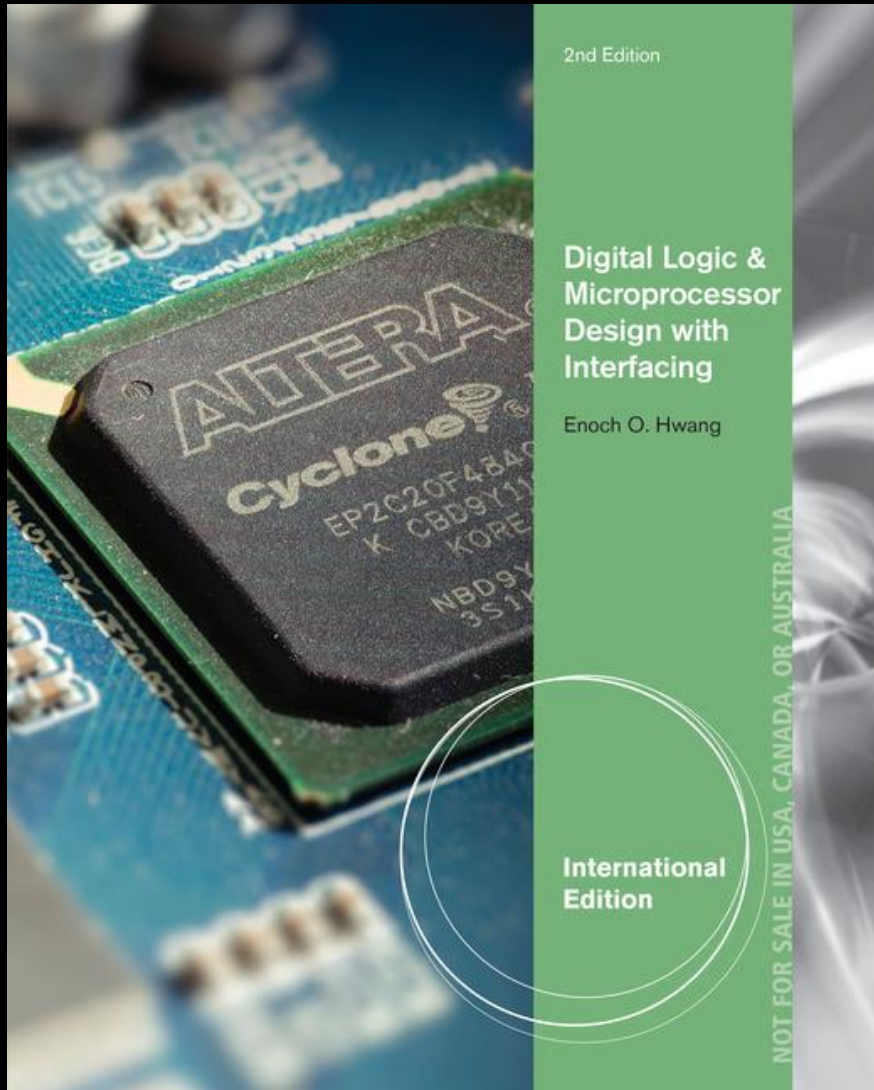
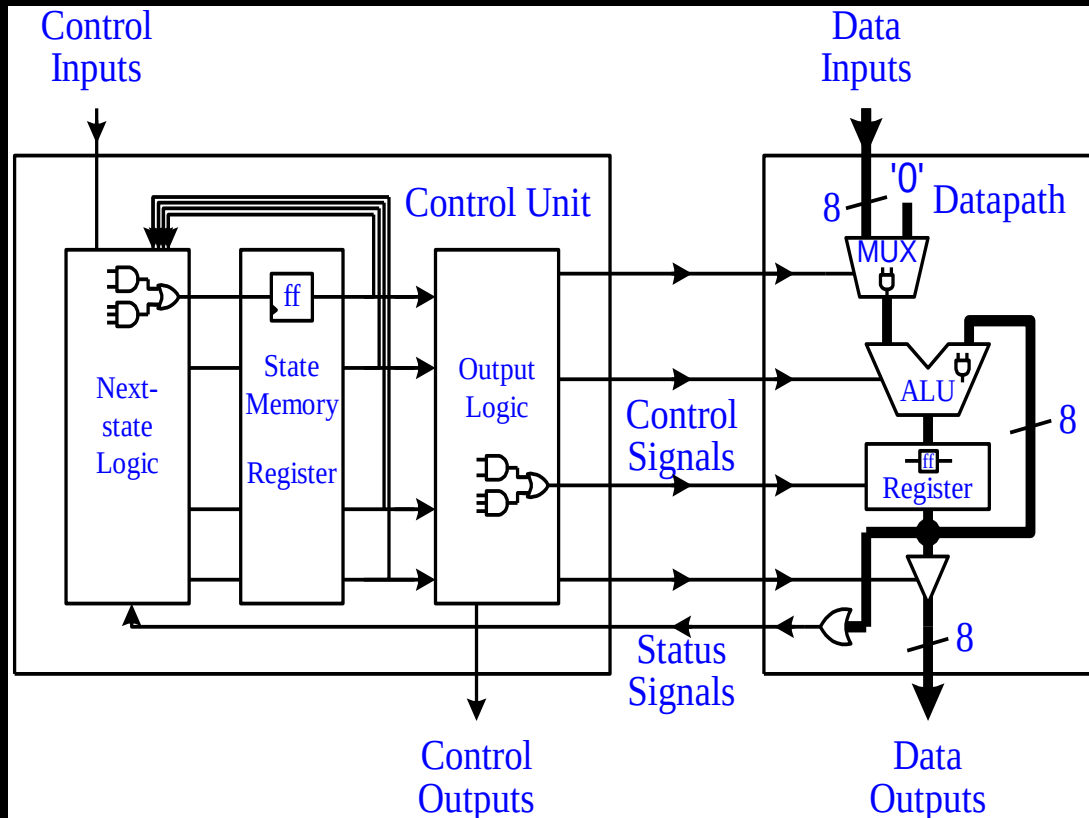




Chapter 4 Standard Combinational Components

Circuit Structure

- Larger circuits are built from smaller blocks



- In Microprocessor:
 - Multiplexer
 - Adder
 - Subtractor
 - ALU
 - Comparator
 - 3-state-buffer
 - Multiplier
 - Next state logic
 - Output logic

In this Chapter

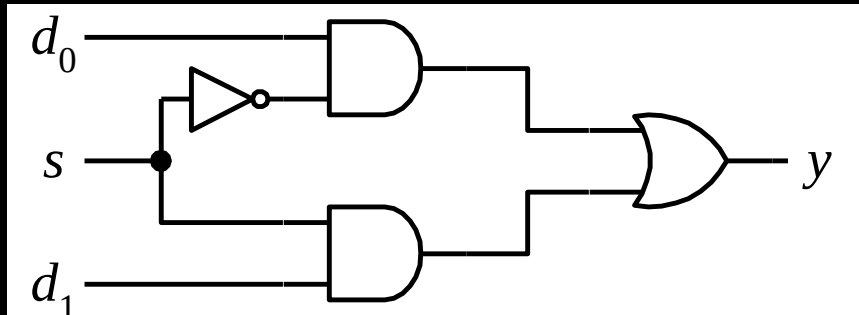
- Multiplexer
- Adder
- Subtractor
- Arithmetic Logic Unit (ALU)
- Decoder
- Tri-state Buffer
- Comparator
- Shifter
- Multiplier

Signal Naming Conventions

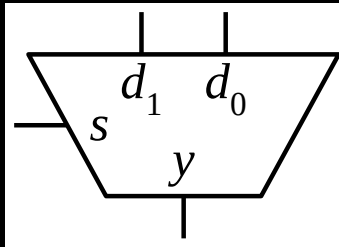
- Positive logic: 1 = on, 0 = off
- Negative logic: 0 = on, 1 = off
- Active-high: 1 active or enabled
- Active-low: 0 active or enabled
- Symbol ' : e.g., *add'* means signal *add* is an active-low signal, i.e., a 0 will perform the add operation
- To assert: means to enable the signal or to make it active
- To de-assert: means to disable the signal or to make it inactive

Multiplexer (Mux)

- n -to-1 multiplexer
 - e.g., 2-1 multiplexer



Circuit



Symbol

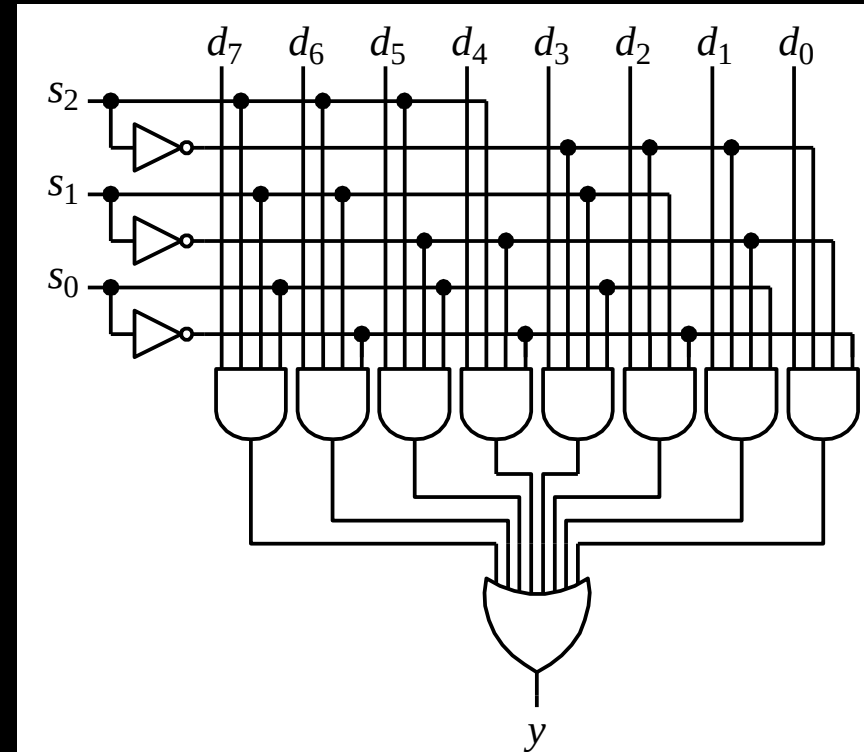
s	d ₁	d ₀	y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Truth table

8-to-1 Multiplexer

Truth Table

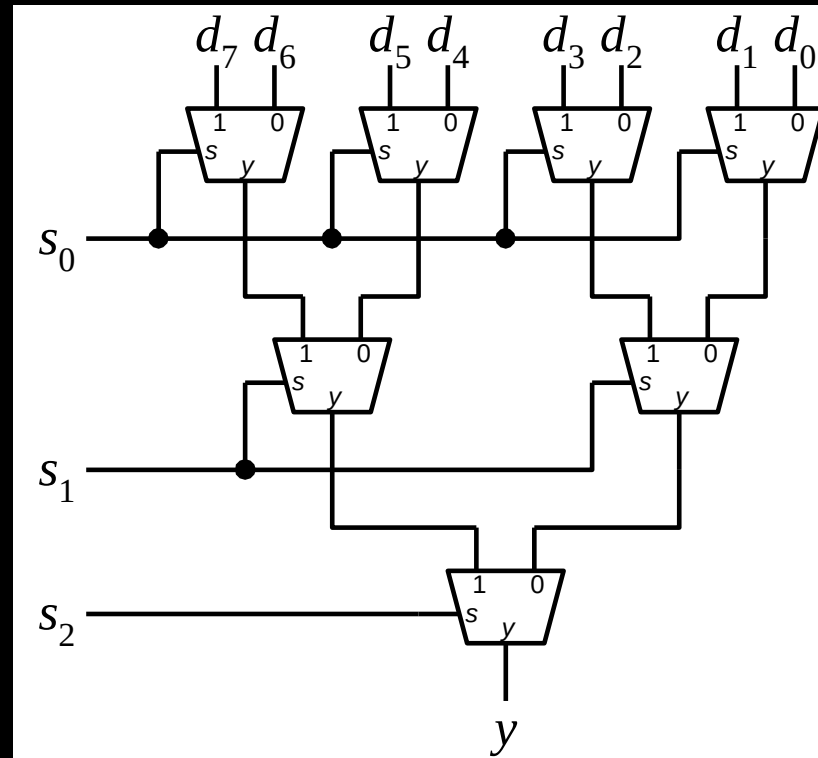
s_2	s_1	s_0	y
0	0	0	d_0
0	0	1	d_1
0	1	0	d_2
0	1	1	d_3
1	0	0	d_4
1	0	1	d_5
1	1	0	d_6
1	1	1	d_7



Circuit

8-to-1 Multiplexer

- Constructed using 2-to-1 muxes



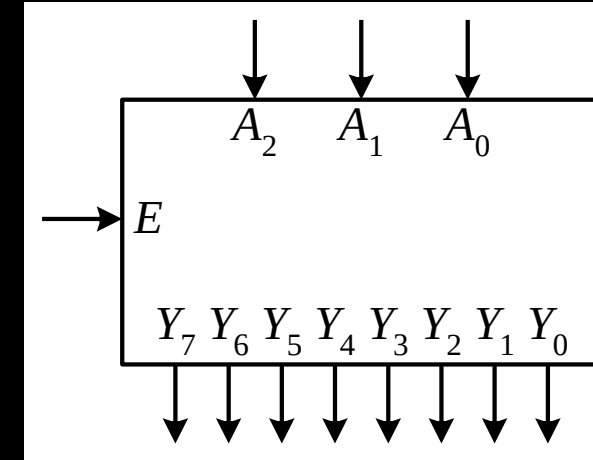
Decoder

- *Decoder (demultiplexer)*
- *m-to-n decoder ($n = 2^m$)*
- *m input lines, $A_{m-1}, \dots, A_0$*
- *n output lines, $Y_{n-1}, \dots, Y_0$*
- *an enable line, E*
- *e.g., 3-to-8 decoder*
- *input address $A = 101$*
 - ➡ *output line Y_5 is asserted*
 - *the rest of the output lines are de-asserted*

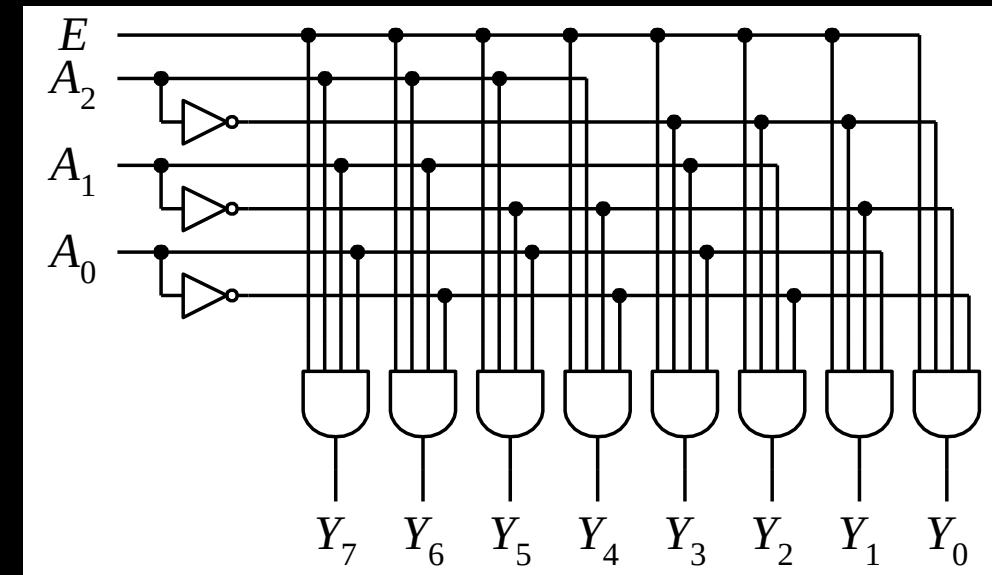
3-to-8 Decoder

E	A ₂	A ₁	A ₀	Y ₇	Y ₆	Y ₅	Y ₄	Y ₃	Y ₂	Y ₁	Y ₀
0	×	×	×	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1
1	0	0	1	0	0	0	0	0	0	1	0
1	0	1	0	0	0	0	0	0	1	0	0
1	0	1	1	0	0	0	0	1	0	0	0
1	1	0	0	0	0	0	1	0	0	0	0
1	1	0	1	0	0	1	0	0	0	0	0
1	1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0

Truth Table



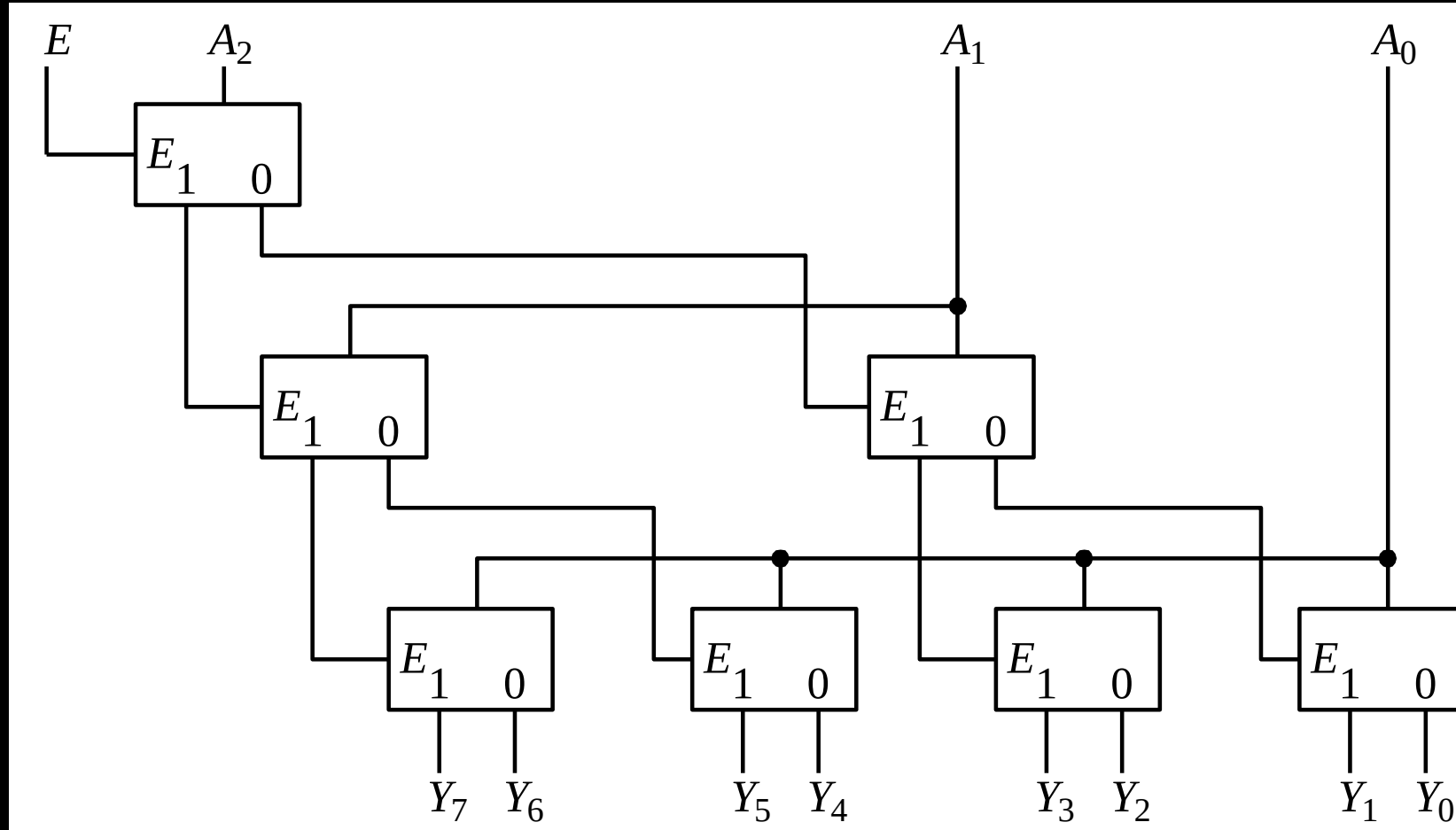
Symbol



Circuit

3-to-8 Decoder

- implemented with seven 1-to-2



Quiz 4.1

Adder

Half-Adder

- **Half-adder**: Adds 2 bits, generates sum and carry
- Design using combinational design process

Step 1: Capture the function

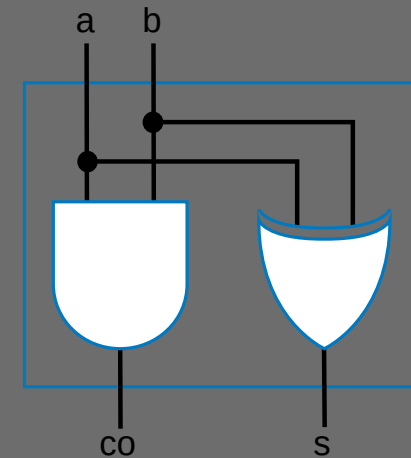
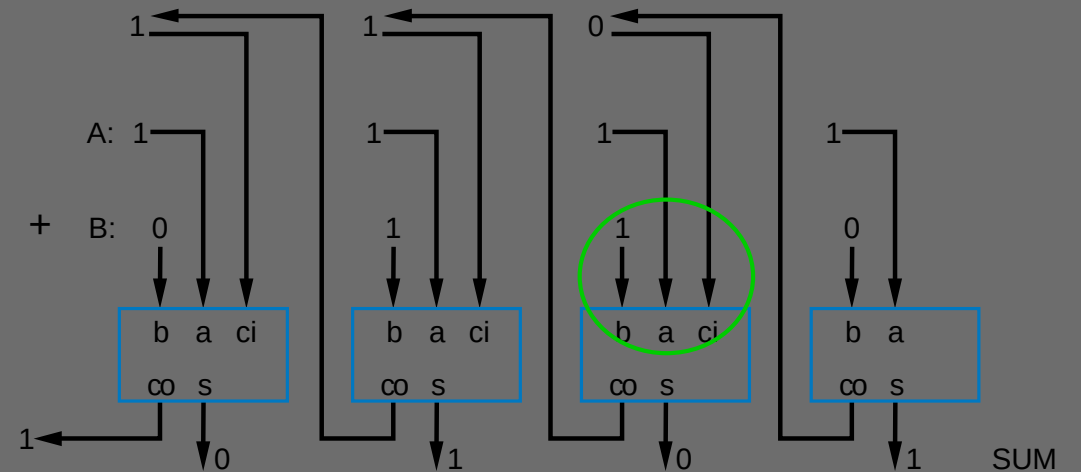
Inputs		Outputs	
a	b	co	s
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Step 2A: Create equations

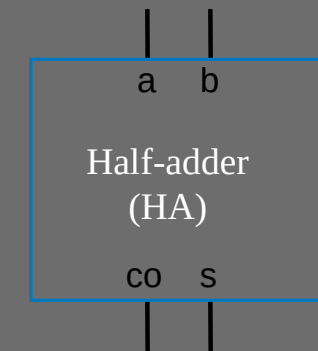
$$co = ab$$

$$s = a'b + ab' \text{ (same as } s = a \text{ xor } b)$$

Step 2B: Implement as circuit



(a)



(b)

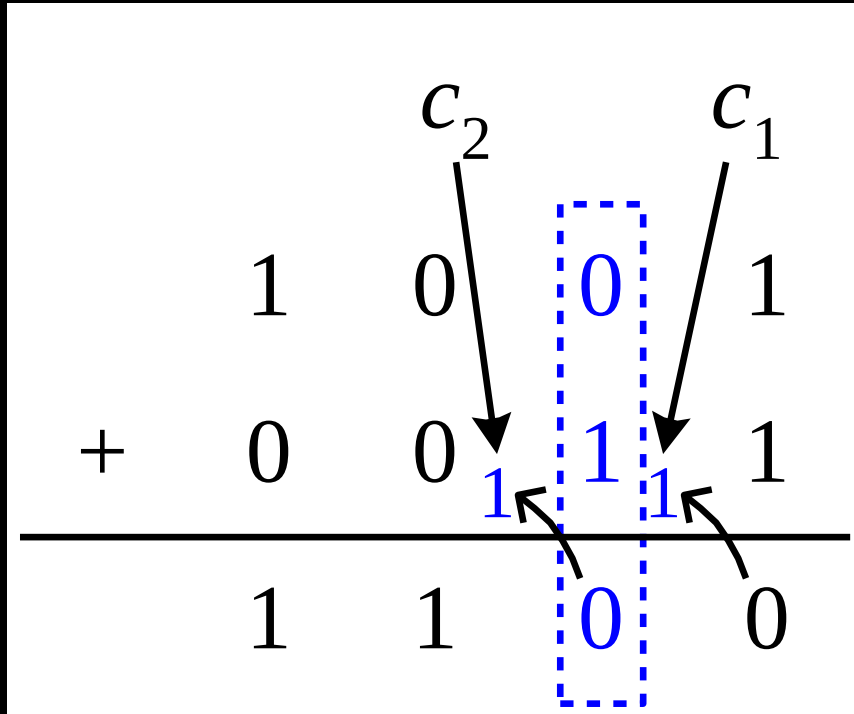
Full Adder

Input:

- Single bit x_i and y_i
- Carry-in bit c_i

Output

- Carry-out c_{i+1}
- Sum bit s_i



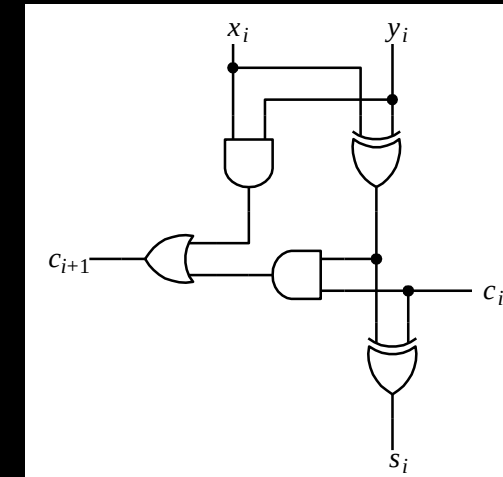
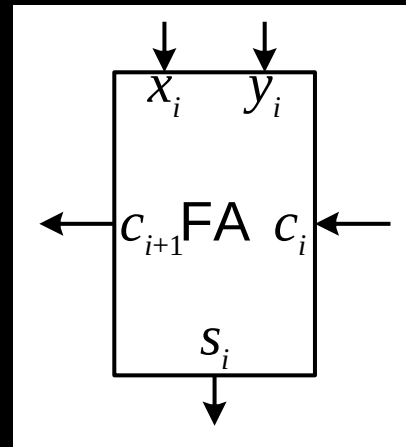
x_i	y_i	c_i	c_{i+1}	s_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Full Adder

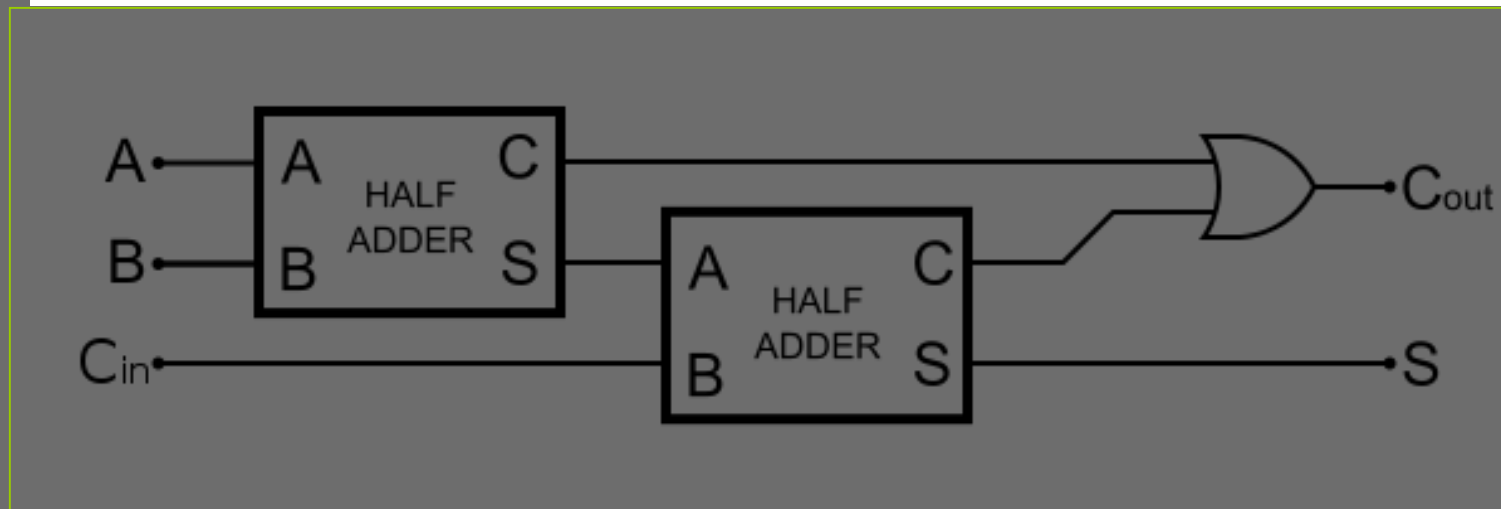
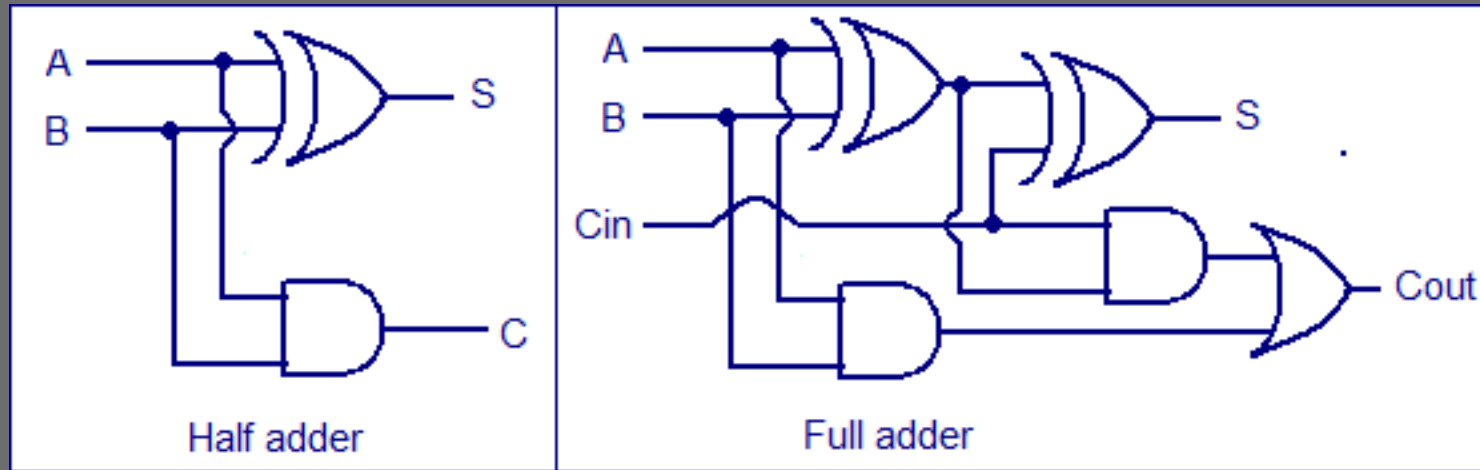
x_i	y_i	c_i	c_{i+1}	s_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$\begin{aligned}
 s_i &= x_i'y_i'c_i + x_i'y_i c_i' + x_i y_i' c_i' + x_i y_i c_i \\
 &= (x_i'y_i + x_i y_i')c_i' + (x_i'y_i' + x_i y_i)c_i \\
 &= (x_i \oplus y_i)c_i' + (x_i \oplus y_i)'c_i \\
 &= x_i \oplus y_i \oplus c_i
 \end{aligned}$$

$$\begin{aligned}
 c_{i+1} &= x_i'y_i c_i + x_i y_i' c_i + x_i y_i c_i' + x_i y_i c_i \\
 &= x_i y_i (c_i' + c_i) + c_i (x_i'y_i + x_i y_i') \\
 &= x_i y_i + c_i (x_i \oplus y_i)
 \end{aligned}$$

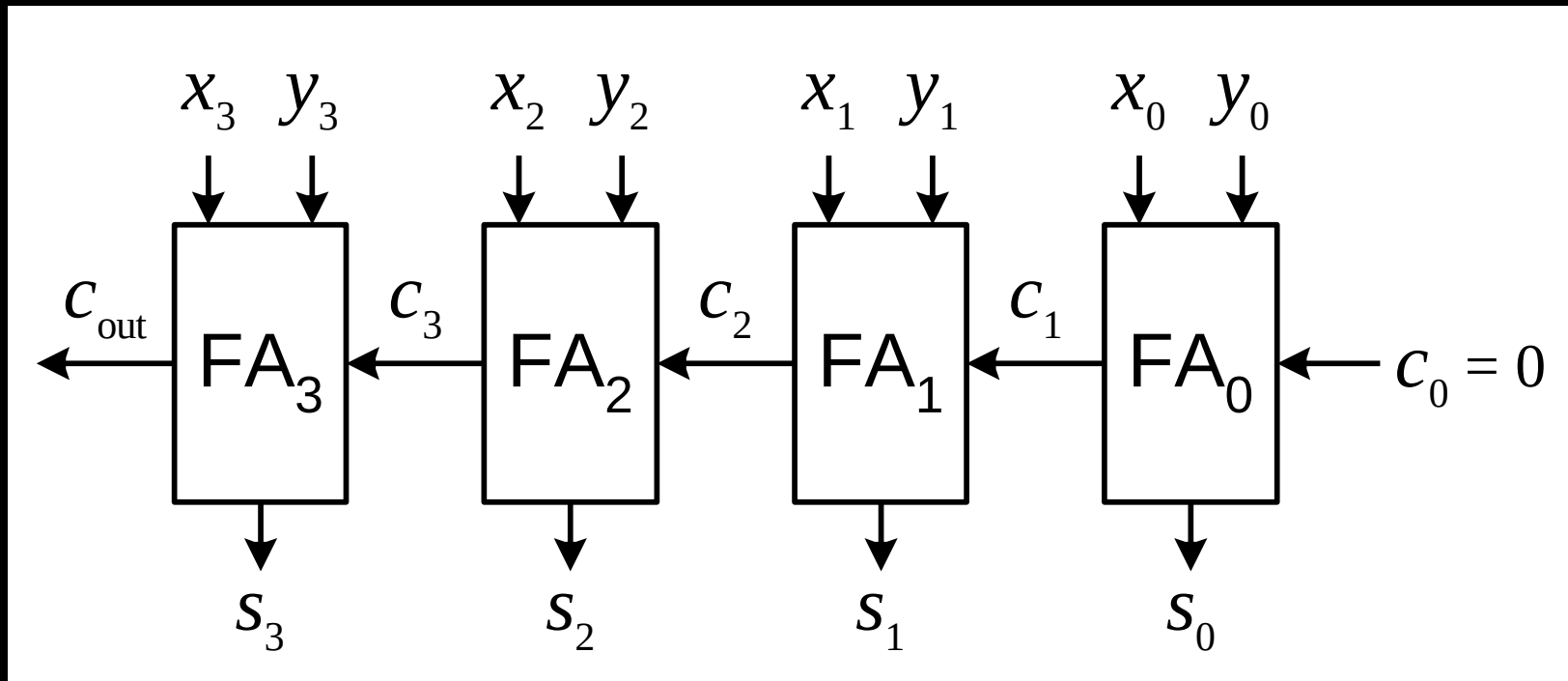


HA and FA



Ripple-Carry Adder

- Adding two 4-bit operands

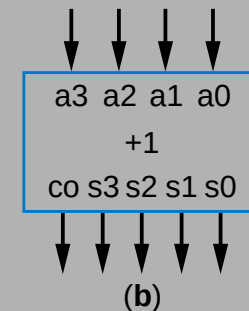
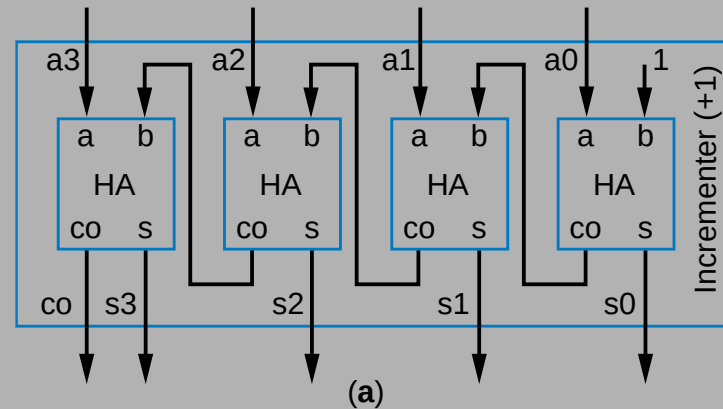


Incrementer

- Adds 1 to input A

Inputs				Outputs				
a3	a2	a1	a0	c0	s3	s2	s1	s0
0	0	0	0	0	0	0	0	1
0	0	0	1	0	0	0	1	0
0	0	1	0	0	0	0	1	1
0	0	1	1	0	0	1	0	0
0	1	0	0	0	0	1	0	1
0	1	0	1	0	0	1	1	0
0	1	1	0	0	0	1	1	1
0	1	1	1	0	1	0	0	0
1	0	0	0	0	1	0	0	1
1	0	0	1	0	1	0	1	0
1	0	1	0	0	1	0	1	1
1	0	1	1	0	1	1	0	0
1	1	0	0	0	1	1	0	1
1	1	0	1	0	1	1	1	0
1	1	1	0	0	1	1	1	1
1	1	1	1	1	0	0	0	0

$$\begin{array}{r}
 \text{carries:} \quad 0 \ 1 \ 1 \\
 \quad \quad \quad 0 \ 0 \ 1 \ 1 \\
 \text{unused} \quad + \quad \text{ } \quad \text{ } \quad \text{ } \quad 1 \\
 \hline
 0 \ 0 \ 1 \ 0 \ 0
 \end{array}$$

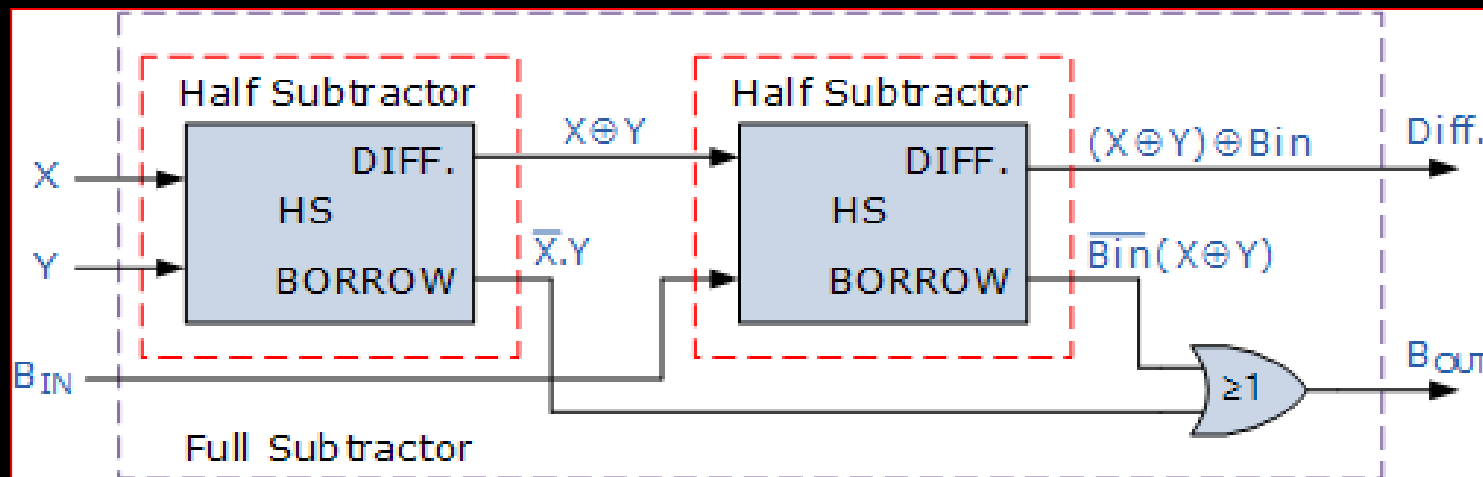
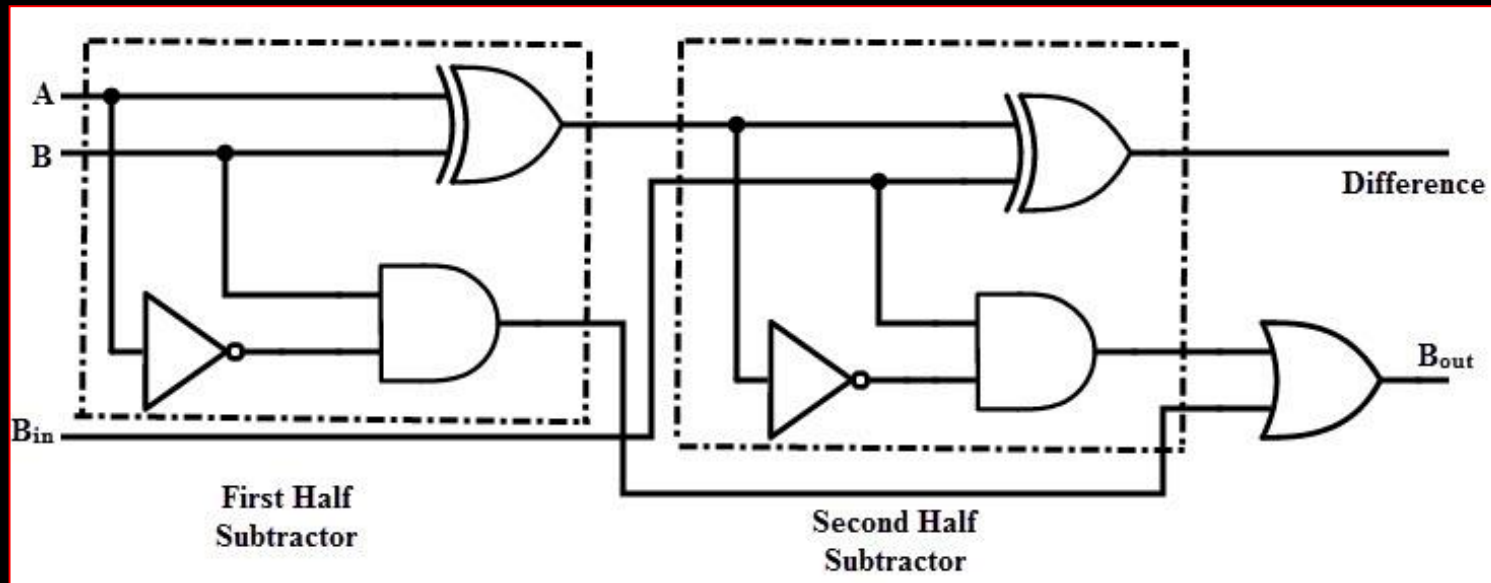


Could design using combinational design process, but smaller design uses carry-ripple, only need half-adders



Subtractor

HS and FS



Subtractor

- one-bit subtractor
 - Similar to full adder
 - sum bit $s_i \longrightarrow$ difference bit d_i
 - carry-in, carry-out signals $\longrightarrow$ borrow-in (b_i) and borrow-out (b_{i+1}) signals
 - $d_i = x_i - b_i + 2b_{i+1} - y_i$

The diagram illustrates a one-bit subtraction operation. It shows a subtraction problem where a minuend (010) is subtracted from a subtrahend (001). The result (001) is shown below a horizontal line. Borrow signals are indicated: b_{i+1} (borrow-in) is 1, and b_i (borrow-out) is 1. A dashed blue box highlights the borrowing process in the middle column, where 0 minus 1 requires borrowing from the left, resulting in 10 minus 1 equals 9 (represented as 0 in binary).

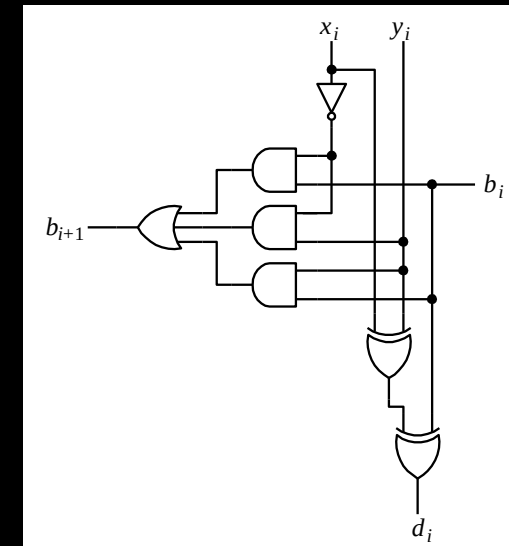
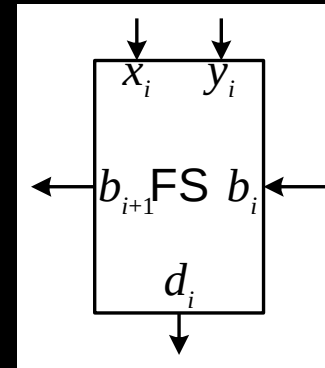
		b_{i+1}		b_i
	0	1	0	0
-	0	0	1	1
	0	0	0	1

Subtractor

x_i	y_i	b_i	b_{i+1}	d_i
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

$$\begin{aligned}
 d_i &= x_i'y_i'b_i + x_i'y_ib_i' + x_iy_i'b_i' + x_iy_ib_i \\
 &= (x_i'y_i + x_iy_i')b_i' + (x_i'y_i' + x_iy_i)b_i \\
 &= (x_i \oplus y_i)b_i' + (x_i \oplus y_i)'b_i \\
 &= x_i \oplus y_i \oplus b_i
 \end{aligned}$$

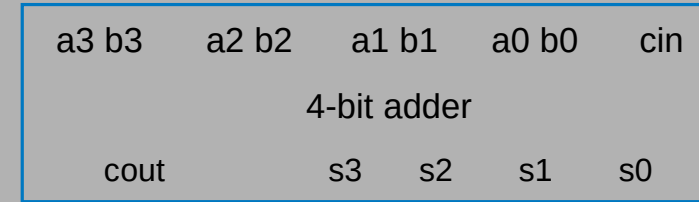
$$\begin{aligned}
 b_{i+1} &= x_i'y_i'b_i + x_i'y_ib_i' + x_i'y_ib_i + x_iy_ib_i \\
 &= x_i'b_i(y_i' + y_i) + x_i'y_i(b_i' + b_i) + y_ib_i(x_i' + x_i) \\
 &= x_i'b_i + x_i'y_i + y_ib_i
 \end{aligned}$$



Faster Adders

Building a Faster Adder

- Built carry-ripple adder
 - Similar to adding by hand, column by column
 - Con: Slow
 - Output is not correct until the carries have rippled to the left – *critical path*
 - 4-bit carry-ripple adder has $4 \times 2 = 8$ gate delays
 - Pro: Small
 - 4-bit carry-ripple adder has just $4 \times 5 = 20$ gates

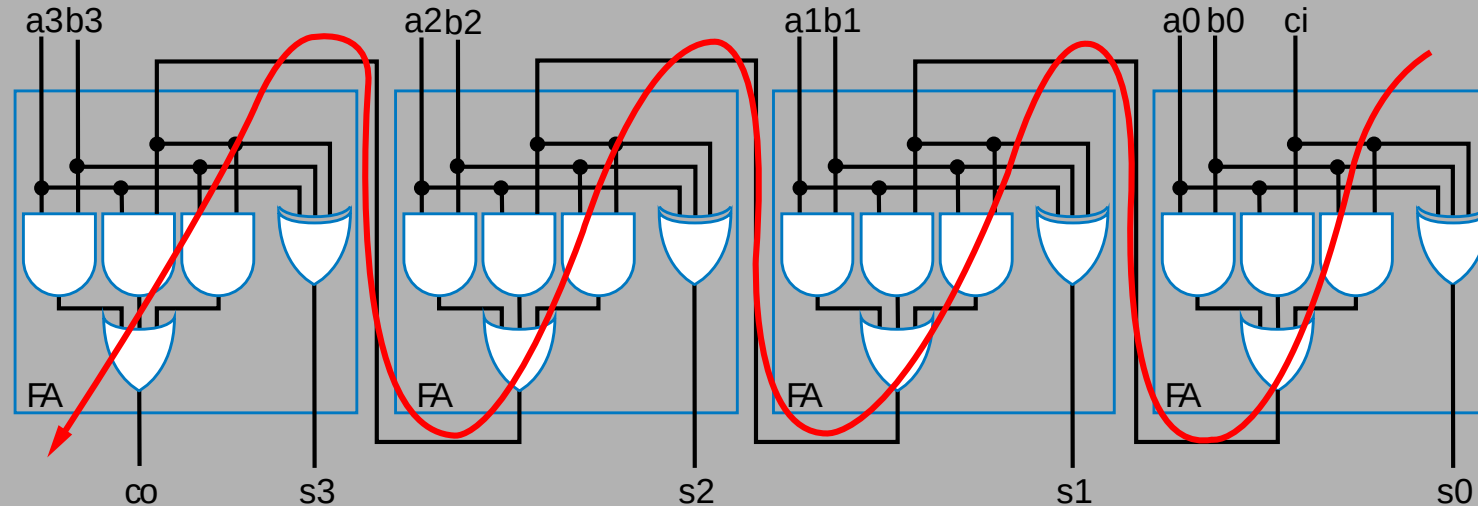


carries: c3 c2 c1 cin

B: b3 b2 b1 b0

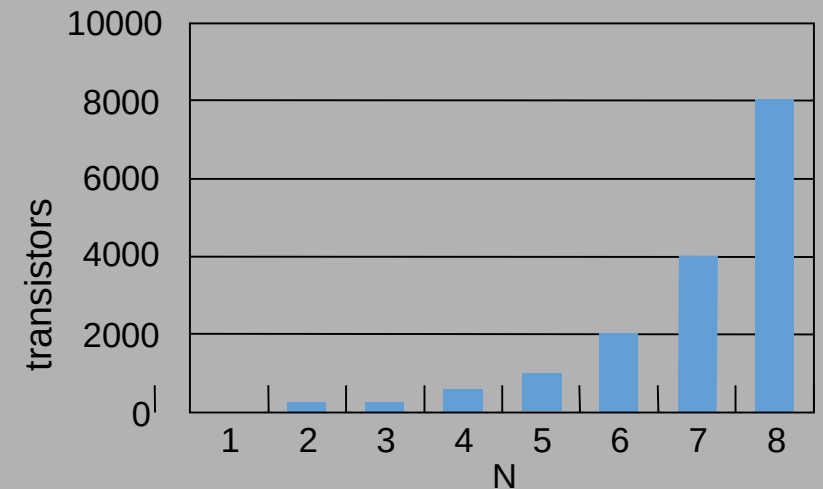
A: + a3 a2 a1 a0

cout s3 s2 s1 s0

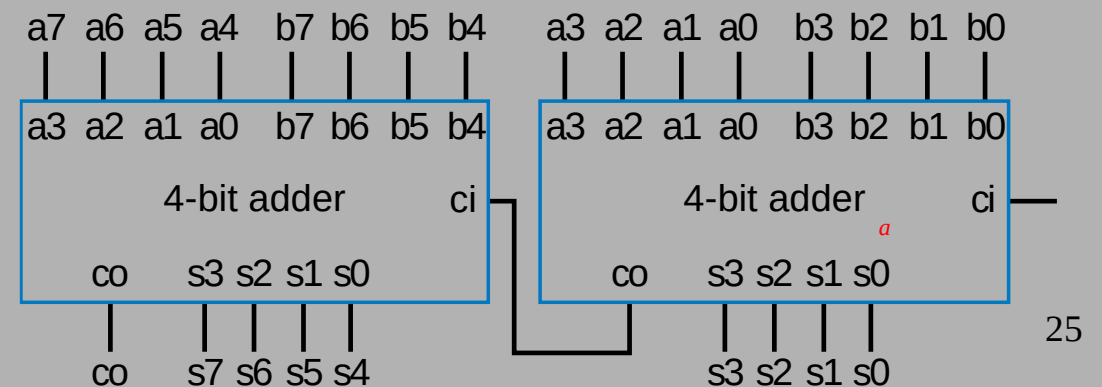


Building a Faster Adder

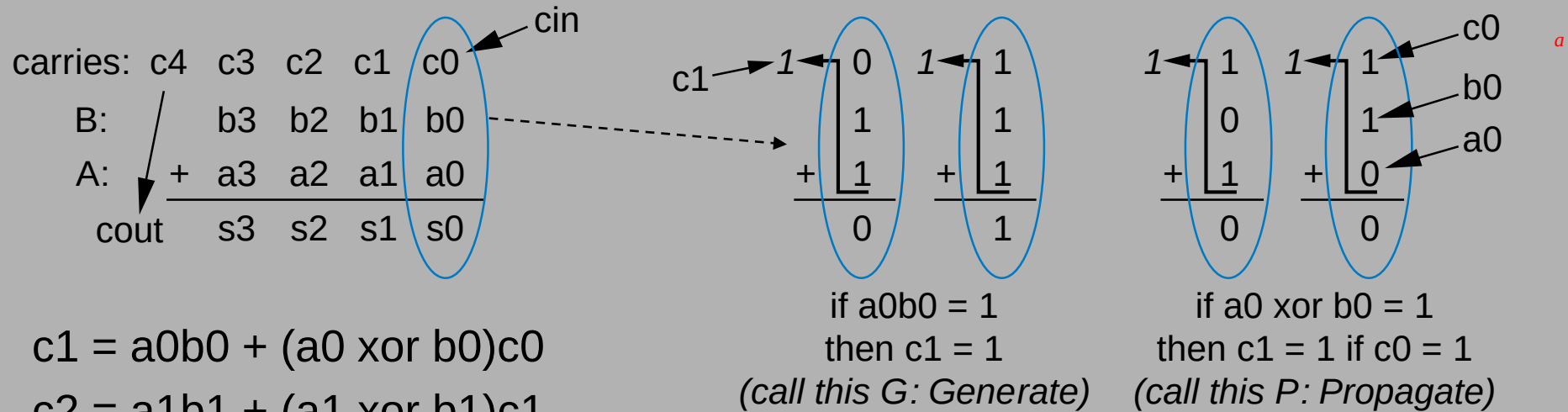
- Carry-ripple is slow but small
 - 8-bit: $8 \times 2 = 16$ gate delays, $8 \times 5 = 40$ gates
 - 16-bit: $16 \times 2 = 32$ gate delays, $16 \times 5 = 80$ gates
 - 32-bit: $32 \times 2 = 64$ gate delays, $32 \times 5 = 160$ gates
- Two-level logic adder (2 gate delays)
 - OK for 4-bit adder: About 100 gates
 - 8-bit: 8,000 transistors / 16-bit: 2 M / 32-bit: 100 billion
 - N-bit two-level adder uses *absurd* number of gates for N much beyond 4
- Compromise
 - Build 4-bit adder using two-level logic, compose to build N-bit adder
 - 8-bit adder: $2 \times (2 \text{ gate delays}) = 4$ gate delays, $2 \times (100 \text{ gates}) = 200$ gates
 - 32-bit adder: $8 \times (2 \text{ gate delays}) = 16$ gate delays, $8 \times (100 \text{ gates}) = 800$ gates



Can we do better?



Efficient Lookahead



$$c_1 = a_0b_0 + (a_0 \text{ xor } b_0)c_0$$

$$c_2 = a_1b_1 + (a_1 \text{ xor } b_1)c_1$$

$$c_3 = a_2b_2 + (a_2 \text{ xor } b_2)c_2$$

$$c_4 = a_3b_3 + (a_3 \text{ xor } b_3)c_3$$

$$c_1 = G_0 + P_0c_0$$

$$c_2 = G_1 + P_1c_1$$

$$c_3 = G_2 + P_2c_2$$

$$c_4 = G_3 + P_3c_3$$

Why those names? When $a_0b_0=1$, we should generate a 1 for c_1 .
When $a_0 \text{ XOR } b_0 = 1$, we should propagate the c_0 value as the value of c_1 , meaning c_1 should equal c_0 .

$G_i = a_i b_i$ (generate)

$P_i = a_i \text{ XOR } b_i$ (propagate)



Efficient Lookahead

- Substituting as in the naïve scheme:

$$c1 = G0 + P0c0$$

$$c2 = G1 + P1c1$$

$$c2 = G1 + P1(G0 + P0c0)$$

$$c2 = G1 + P1G0 + P1P0c0$$

$$c3 = G2 + P2c2$$

$$c3 = G2 + P2(G1 + P1G0 + P1P0c0)$$

$$c3 = G2 + P2G1 + P2P1G0 + P2P1P0c0$$

$$c4 = G3 + P3G2 + P3P2G1 + P3P2P1G0 + P3P2P1P0c0$$

$G_i = a_i b_i$ (generate)

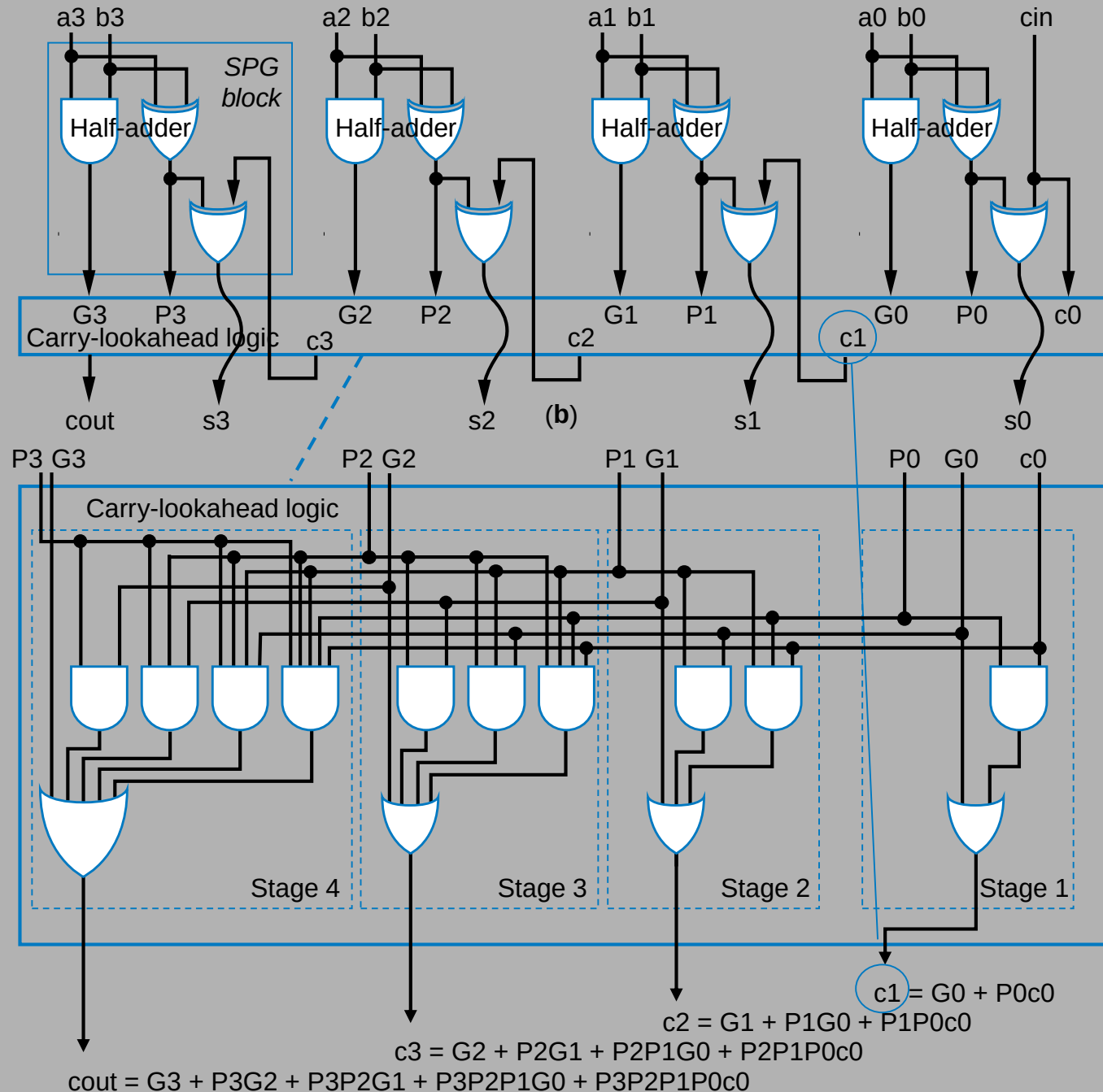
$P_i = a_i \text{ XOR } b_i$ (propagate)

G_i/P_i function of inputs only

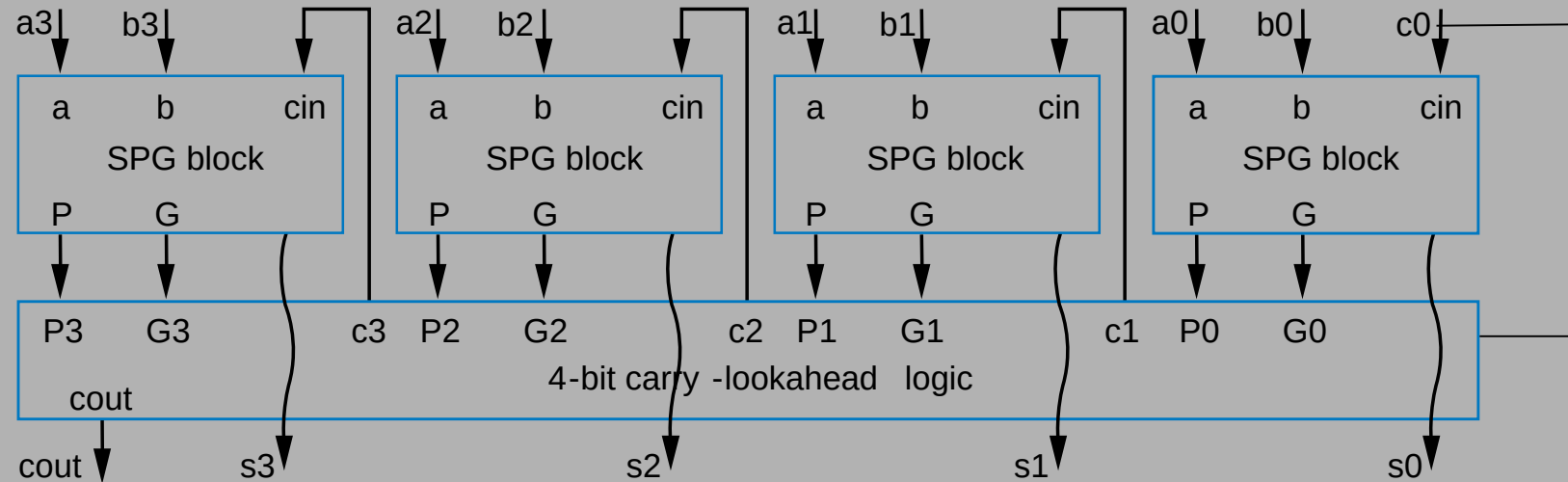


CLA

- Each stage:
 - HA for G and P
 - Another XOR for s
 - Call SPG block
- Create carry-lookahead logic from equations
- More efficient than naïve scheme, at expense of one extra gate delay



Carry-Lookahead Adder – High-Level View

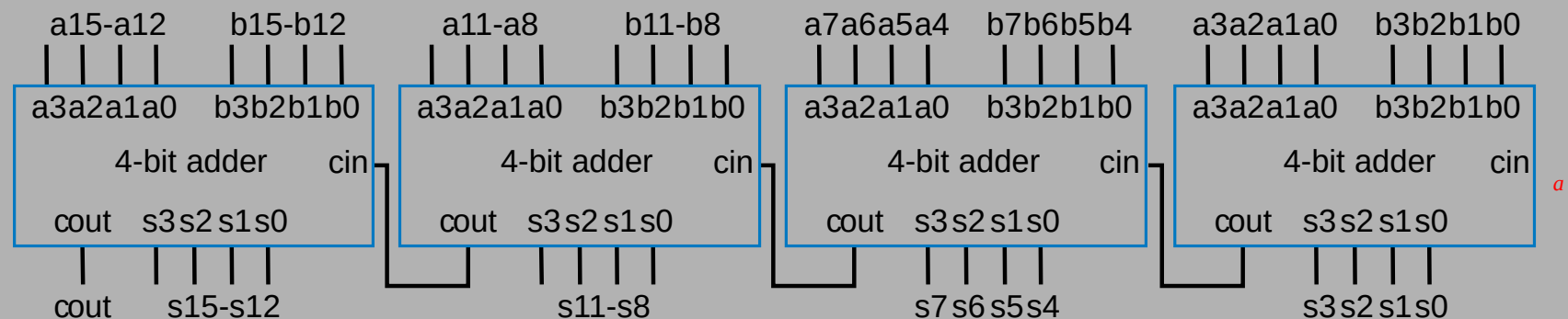
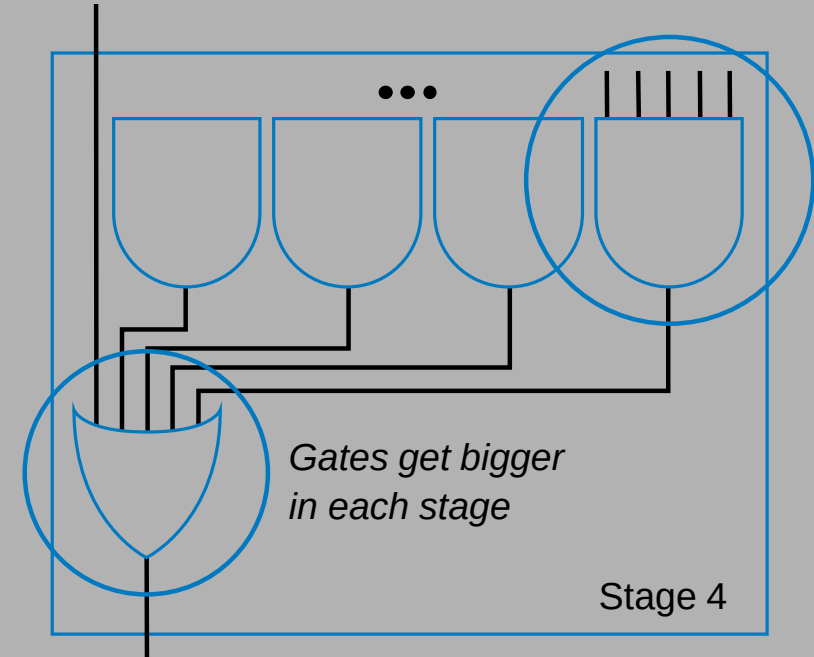


- Fast – only **4 gate delays**
 - Each stage has SPG block with 2 gate levels
 - Carry-lookahead logic quickly computes the carry from the propagate and generate bits using 2 gate levels inside
- Reasonable number of gates – 4-bit adder has only **26 gates**
- 4-bit adder comparison (gate delays, gates)
 - Carry-ripple: (8, 20)
 - Two-level: (2, 500)
 - CLA: (4, 26)
 - o **Nice compromise**



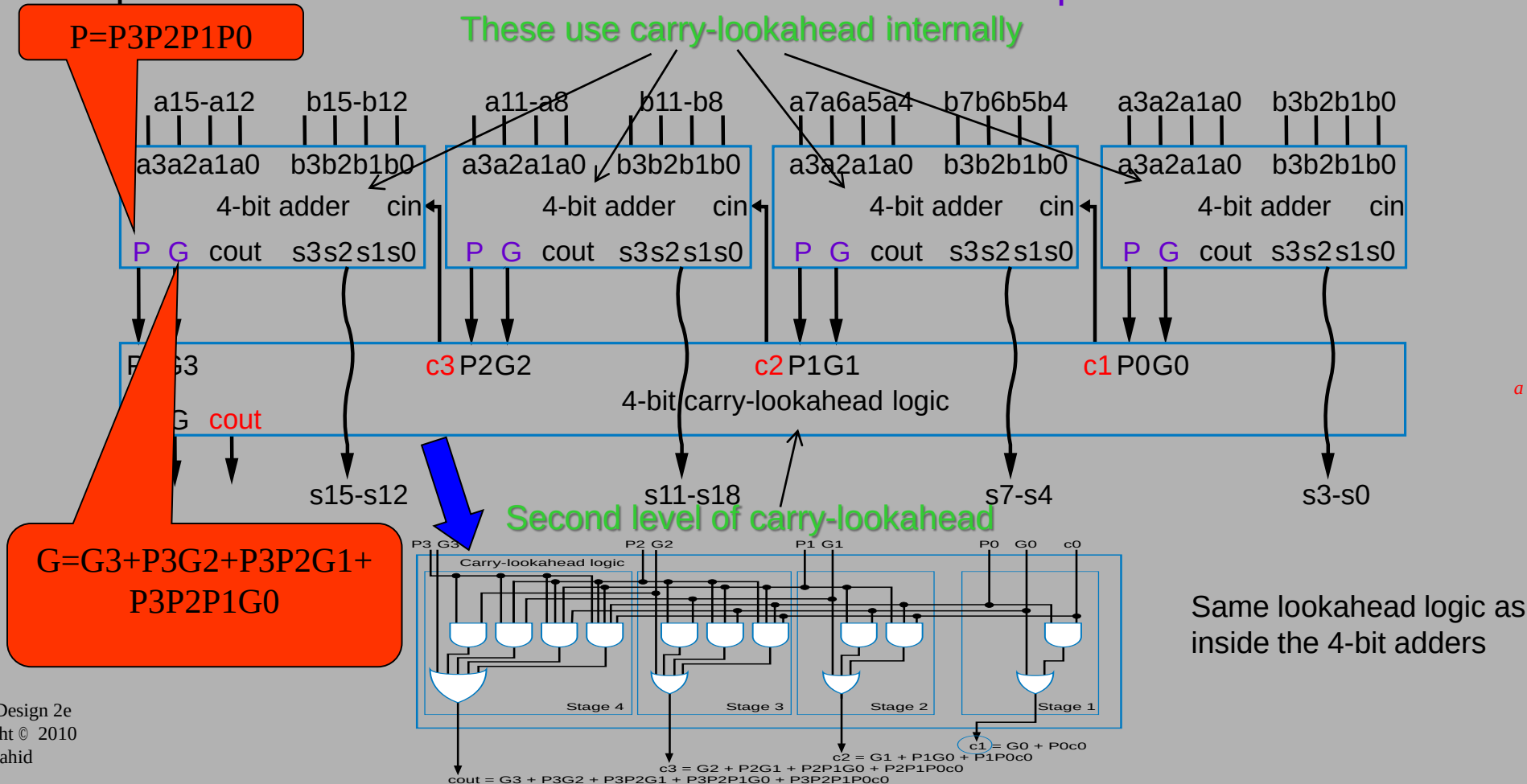
Carry-Lookahead Adder – 32-bit?

- Problem: Gates get bigger in each stage
 - 4th stage has 5-input gates
 - 32nd stage would have 33-input gates
 - Too many inputs for one gate
 - Would require building from smaller gates, meaning more levels (slower), more gates (bigger)
- One solution: Connect 4-bit CLA adders in carry-ripple manner
 - Ex: 16-bit adder: $4 + 4 + 4 + 4 = 16$ gate delays.
Can we do better?

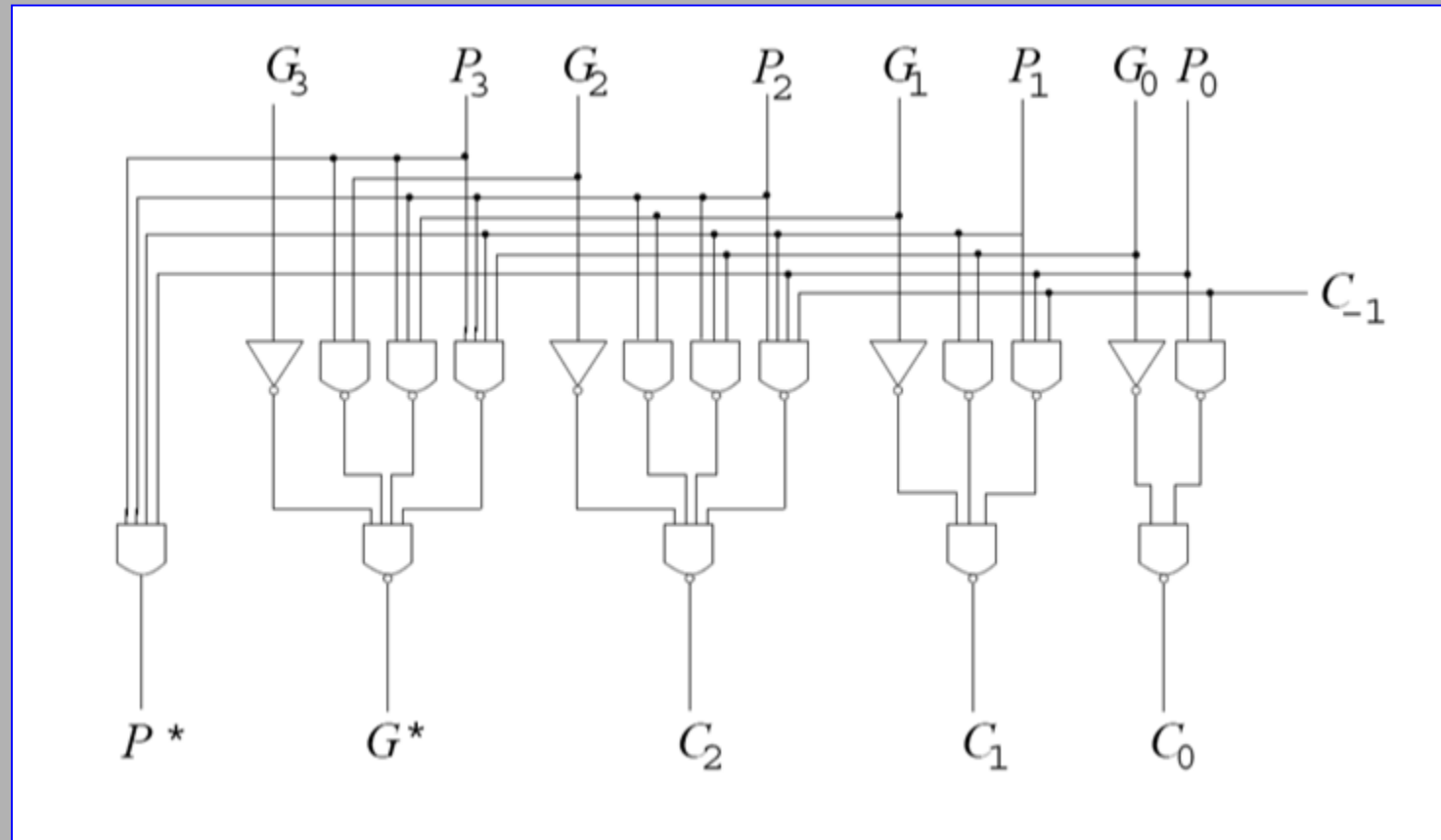


Hierarchical Carry-Lookahead Adders

- Better solution – Rather than rippling the carries, just *repeat* the carry-lookahead concept
 - Requires minor modification of 4-bit CLA adder to **output P and G**



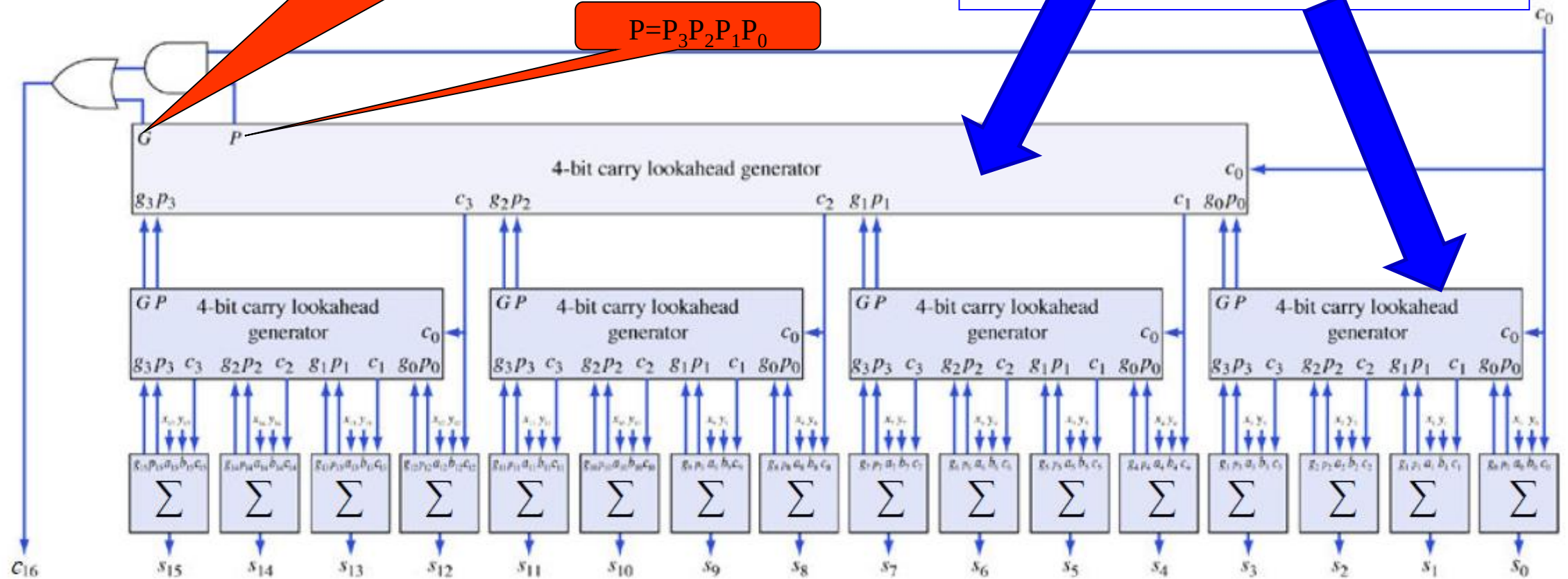
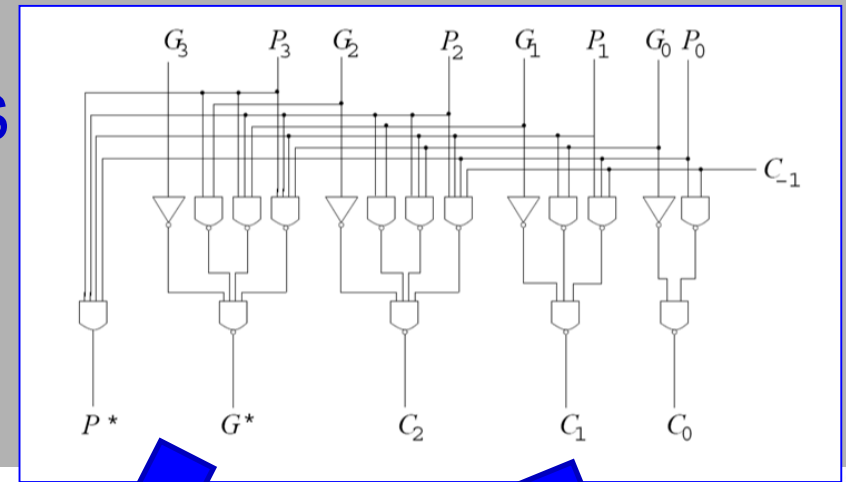
Carry Look-ahead Generator



Hierarchical Carry-Lookahead Adders

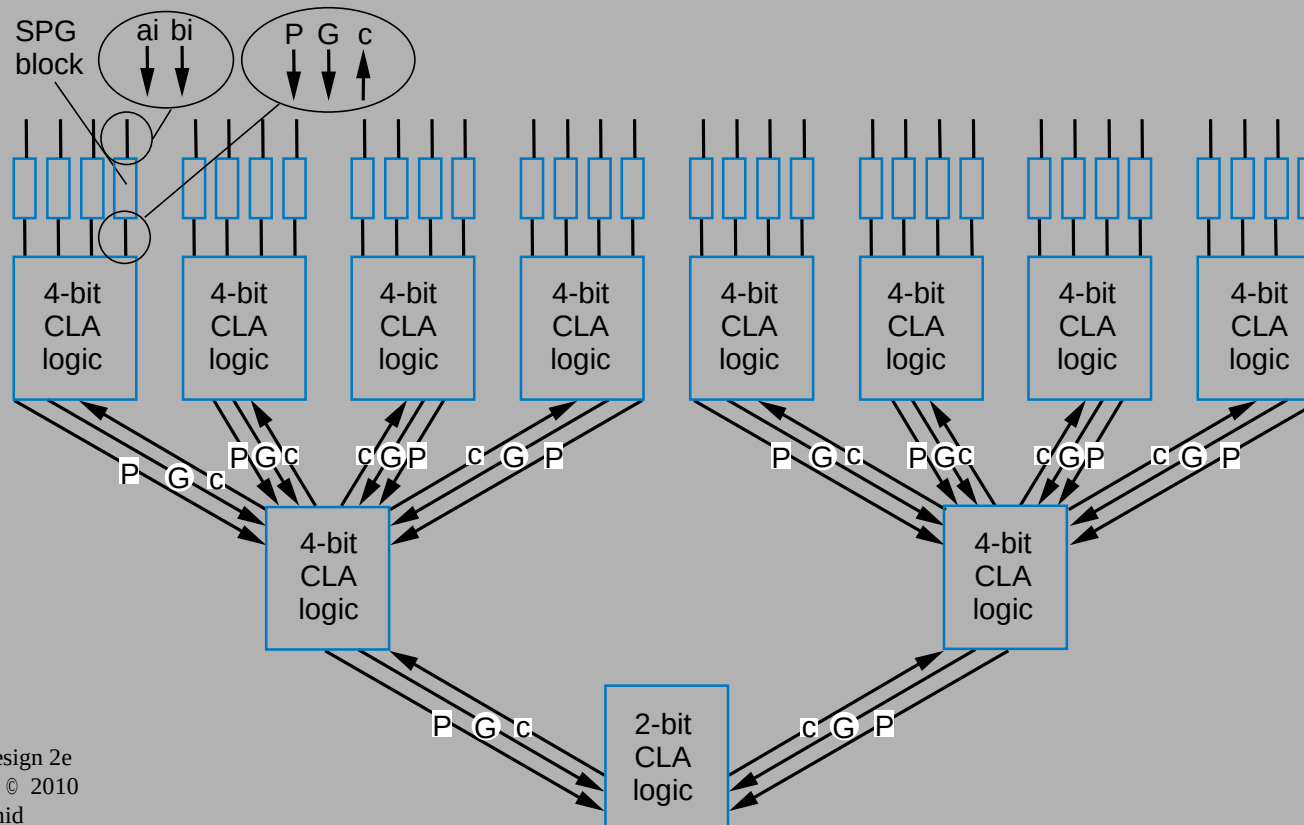
$$G = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0$$

$$P = P_3 P_2 P_1 P_0$$



Hierarchical Carry-Lookahead Adders

- Hierarchical CLA concept can be applied for larger adders
- 32-bit hierarchical CLA:
 - Only about 8 gate delays (2 for SPG block, then 2 for 1st level, 2 for 2nd level, 2 for 3rd level CLA)
 - Only about 14 gates in each 4-bit CLA logic block



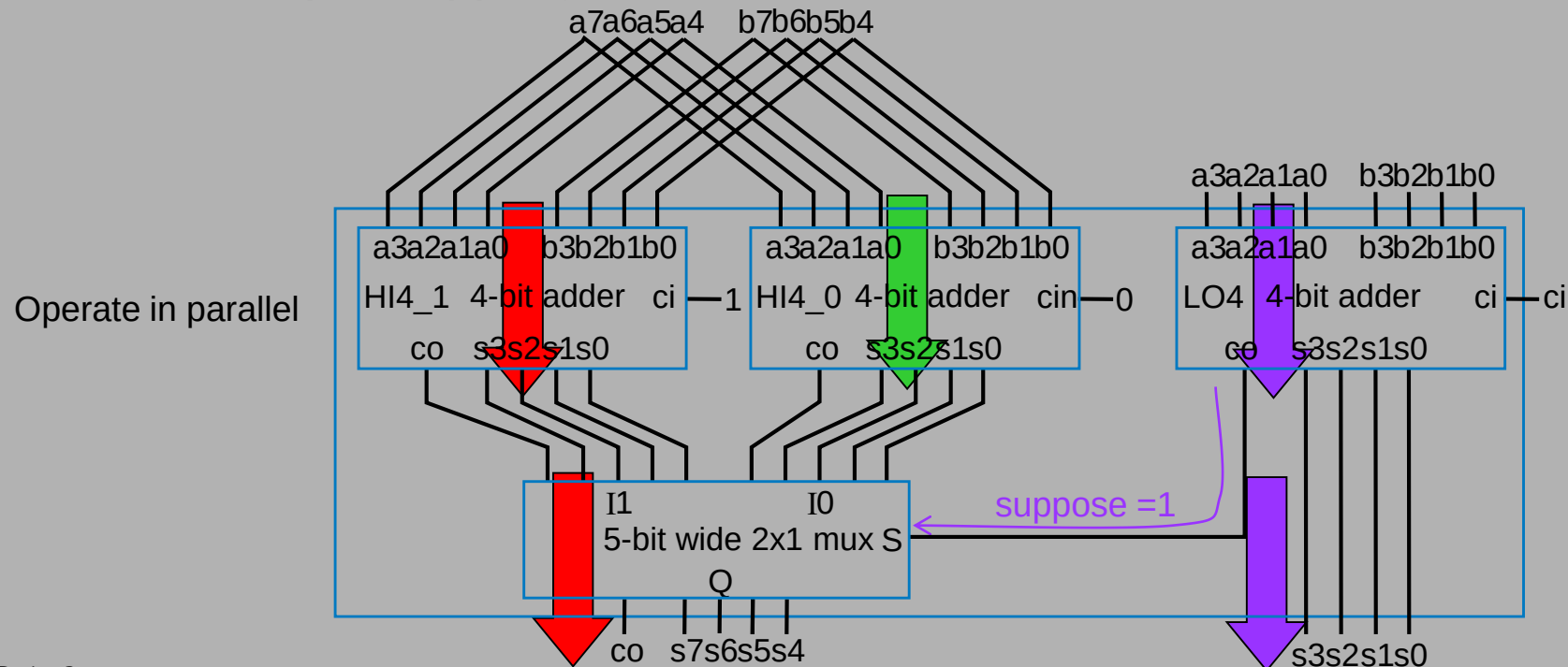
Q: How many gate delays for 64-bit hierarchical CLA, using 4-bit CLA logic?

A: 16 CLA-logic blocks in 1st level, 4 in 2nd, 1 in 3rd -- so still just 8 gate delays (2 for SPG, and 2+2+2 for CLA logic). *CLA is a very efficient method.*

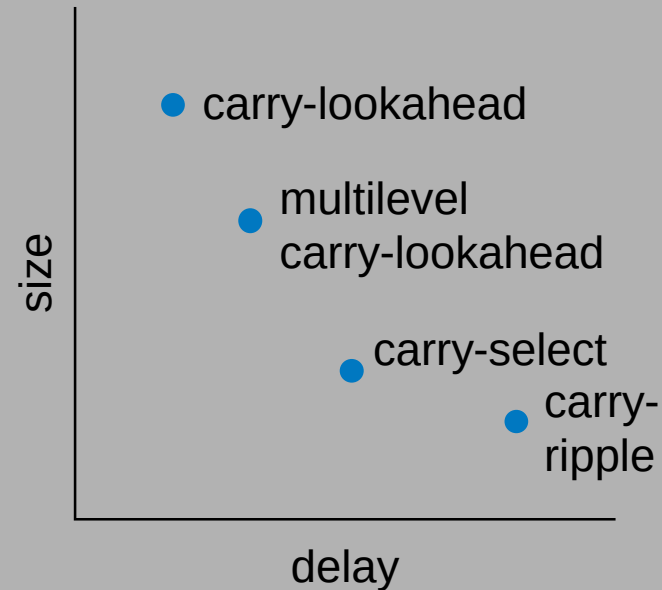


Carry Select Adder

- Another way to compose adders
 - High-order stage – Compute result for carry in of **1** and of **0**
 - Select based on **carry-out** of low-order stage
 - Faster than pure rippling



Adder Tradeoffs



- Designer picks the adder that satisfies particular delay and size requirements
 - May use different adder types in different parts of same design
 - Faster adders on critical path, smaller adders on non-critical path



Quiz 4.2

Adder-Subtractor Combination

- Based on ripple-carry adder
- Subtraction: adding the negated value of the second operand
 - $A - B \longrightarrow A + (-B)$
 - To get $-B$, use two's complement:
 - i.e., invert B and then add a 1
 - addition operation
 - do not flip the bits and don't add an extra 1
 - subtraction operation
 - flip the bits of the B operand and then add an extra 1

Adder-Subtractor Combination

- Use a select signal s to select whether to add or subtract

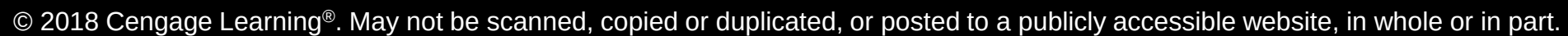
s	Function	Operation
0	Add	$F = A + B$
1	Subtract	$F = A + B' + 1$

s	b_i	y_i	c_0
0	0	0	0
0	1	1	0
1	0	1	1
1	1	0	1

Truth table for y_i and c_0

- What is the circuit for y_i and c_0 ?

The diagram illustrates a 4-bit ripple-carry adder. It consists of four Full Adders (FA) connected in series. The inputs are two 4-bit numbers, $a_3a_2a_1a_0$ and $b_3b_2b_1b_0$, and an initial carry-in s . The carry-out of each FA is the carry-in for the next higher bit. The final carry-out is c_{out} . The outputs are the 4-bit sum $f_3f_2f_1f_0$. Overflow detection is implemented using two 3-input OR gates. The *Unsigned Overflow* gate takes c_{out} , y_3 , and y_2 as inputs. The *Signed Overflow* gate takes c_{out} , y_3 , and y_0 as inputs. The carry propagation signals y_i are the XOR of a_i and b_i .



Detecting Overflow: Method 1

- For two's complement numbers, overflow occurs when the two numbers' sign bits are the same but differ from the result's sign bit
 - If the two numbers' sign bits are initially different, overflow is impossible
 - Adding positive and negative can't exceed largest magnitude positive or negative
- Simple overflow detection circuit for 4-bit adder
 - $\text{overflow} = a_3'b_3'f_3 + a_3b_3f_3'$
 - Include "overflow" output bit on adder/subtractor

sign bits

$\begin{array}{r} 011 \\ + 0001 \\ \hline 1000 \end{array}$ overflow (a)	$\begin{array}{r} 1111 \\ + 1000 \\ \hline 0111 \end{array}$ overflow (b)	$\begin{array}{r} 1000 \\ + 0111 \\ \hline 1111 \end{array}$ no overflow (c)
---	--	---

If the numbers' sign bits have the same value, which differs from the result's sign bit, overflow has occurred.

Detecting Overflow: Method 2

- Even simpler method: Detect difference between carry-in to sign bit and carry-out from sign bit
- Yields simpler circuit: $\text{overflow} = c_3 \text{ xor } c_4$

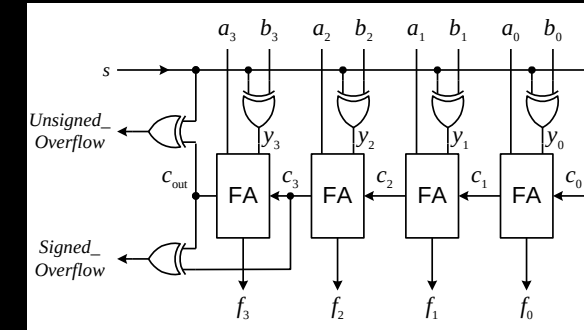
1 1 1	0 0 0	0 0 0
0 1 1 1	1 1 1 1	1 0 0 0
+ 0 0 0 1	+ 1 0 0 0	+ 0 1 1 1
0 1 0 0 0	1 0 1 1 1	0 1 1 1 1
overflow	overflow	no overflow
(a)	(b)	(c)

If the carry into the sign bit column differs from the carry out of that column, overflow has occurred.

Unsigned & Signed Overflow

- *adder-subtractor combination*

- Unsigned_Overflow (a)
- Signed_Overflow (a)

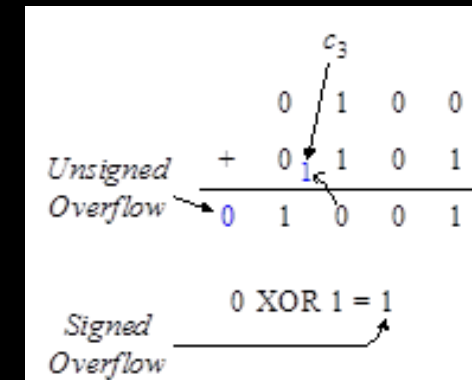


(a)

- *e.g., 4-bit signed number*

(-2^3 to $2^3 - 1$)

- (b) $4 + 5 = 9 \rightarrow$ signed overflow = 1
unsigned overflow = 0

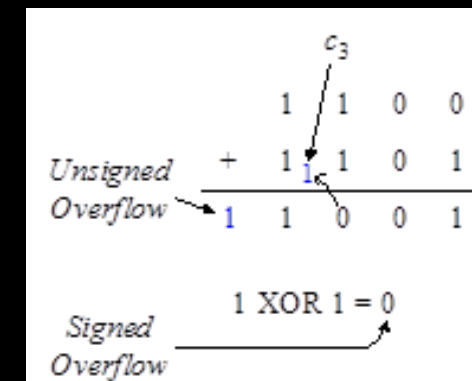


(b)

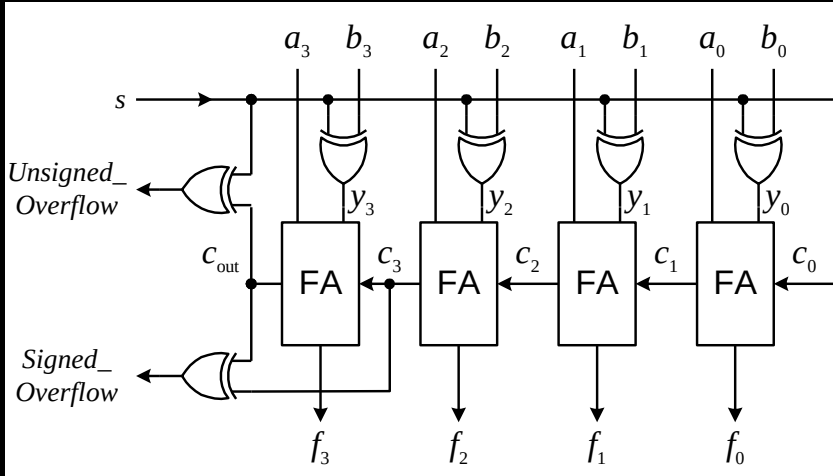
- (c) $-4 + (-3) = -7$

$\rightarrow$ signed overflow = 0

unsigned overflow = 1 (12+13=25)



(c)



	4-bit Calculations	Figure 4.14 Circuit	Signed Extend	Sign Overflow	Zero Extend	Unsigned Overflow
a.	$\begin{array}{r} 0011 \\ + 00100 \\ \hline 0111 \end{array}$	$\begin{array}{r} 0011 \\ + 0100 \\ \hline 00111 \end{array}$	$\begin{array}{r} 00011 \\ + 00100 \\ \hline 00111 \end{array}$	0	$\begin{array}{r} 00011 \\ + 00100 \\ \hline 00111 \end{array}$	0
b.	$\begin{array}{r} 1111 \\ + 1110 \\ \hline 11101 \end{array}$	$\begin{array}{r} 1111 \\ + 1110 \\ \hline 11101 \end{array}$	$\begin{array}{r} 11111 \\ + 11110 \\ \hline 11101 \end{array}$	0	$\begin{array}{r} 01111 \\ + 01110 \\ \hline 11101 \end{array}$	1
c.	$\begin{array}{r} 0111 \\ + 01110 \\ \hline 1101 \end{array}$	$\begin{array}{r} 0111 \\ + 0110 \\ \hline 01101 \end{array}$	$\begin{array}{r} 00111 \\ + 00110 \\ \hline 01101 \end{array}$	1	$\begin{array}{r} 00111 \\ + 00110 \\ \hline 01101 \end{array}$	0
d.	$\begin{array}{r} 1000 \\ + 1001 \\ \hline 10001 \end{array}$	$\begin{array}{r} 1000 \\ + 1001 \\ \hline 10001 \end{array}$	$\begin{array}{r} 11000 \\ + 11001 \\ \hline 10001 \end{array}$	1	$\begin{array}{r} 01000 \\ + 01001 \\ \hline 10001 \end{array}$	1
e.	$\begin{array}{r} 0111 \\ - 00110 \\ \hline 0001 \end{array}$	$\begin{array}{r} 0111 \\ + 1001_1 \\ \hline 10001 \end{array}$	$\begin{array}{r} 00111 \\ - 00110 \\ \hline 00001 \end{array}$	0	$\begin{array}{r} 00111 \\ - 00110 \\ \hline 00001 \end{array}$	0
f.	$\begin{array}{r} 0000 \\ - 0111 \\ \hline 11001 \end{array}$	$\begin{array}{r} 0000 \\ + 1000_1 \\ \hline 01001 \end{array}$	$\begin{array}{r} 00000 \\ - 00111 \\ \hline 11001 \end{array}$	0	$\begin{array}{r} 00000 \\ - 00111 \\ \hline 11001 \end{array}$	1
g.	$\begin{array}{r} 1000 \\ - 01001 \\ \hline 0111 \end{array}$	$\begin{array}{r} 1000 \\ + 1110_1 \\ \hline 10111 \end{array}$	$\begin{array}{r} 11000 \\ - 00001 \\ \hline 10111 \end{array}$	1	$\begin{array}{r} 01000 \\ - 00001 \\ \hline 00111 \end{array}$	0
h.	$\begin{array}{r} 0001 \\ - 10000 \\ \hline 11001 \end{array}$	$\begin{array}{r} 0001 \\ + 0111_1 \\ \hline 01001 \end{array}$	$\begin{array}{r} 00001 \\ - 11000 \\ \hline 01001 \end{array}$	1	$\begin{array}{r} 00001 \\ - 01000 \\ \hline 11001 \end{array}$	1

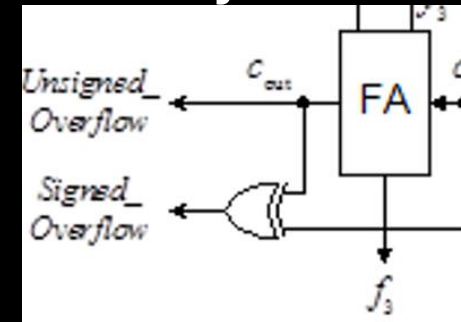
- 1) In the Zero Extend column, the most significant bit (highlighted in blue) is already the correct unsigned overflow bit.
- 2) In the Signed Extend column, XORing the two most significant bits in the result (highlighted in blue) is the correct signed overflow bit.
- 3) In the 4-bit Calculations column, the four cases with the blue highlights are exactly where the signed overflow is a 1 for the Boolean equations:

$$\text{Signed_Overflow} = (A_n' \cdot B_n' \cdot F_n) + (A_n \cdot B_n \cdot F_n')$$

for additions

$$\text{Signed_Overflow} = (A_n \cdot B_n' \cdot F_n') + (A_n' \cdot B_n \cdot F_n)$$

for subtractions



Notice that this is also correct if we apply these Boolean equations to bit 4 of A , B , and F in the Zero Extended calculations since bit 4 in these 5-bit calculations is the most significant bit in the original calculations.

- 4) In the Circuit column, the circuit does not perform subtractions. Instead, it performs the addition of $A+B'$ plus a 1 for c_0 . For addition, the most significant bit (highlighted in blue) is already the correct unsigned overflow bit. However, for subtraction, the most significant bit (highlighted in blue) is the inverse of the unsigned overflow bit. Hence, we need to XOR the overflow bit with the select signal.

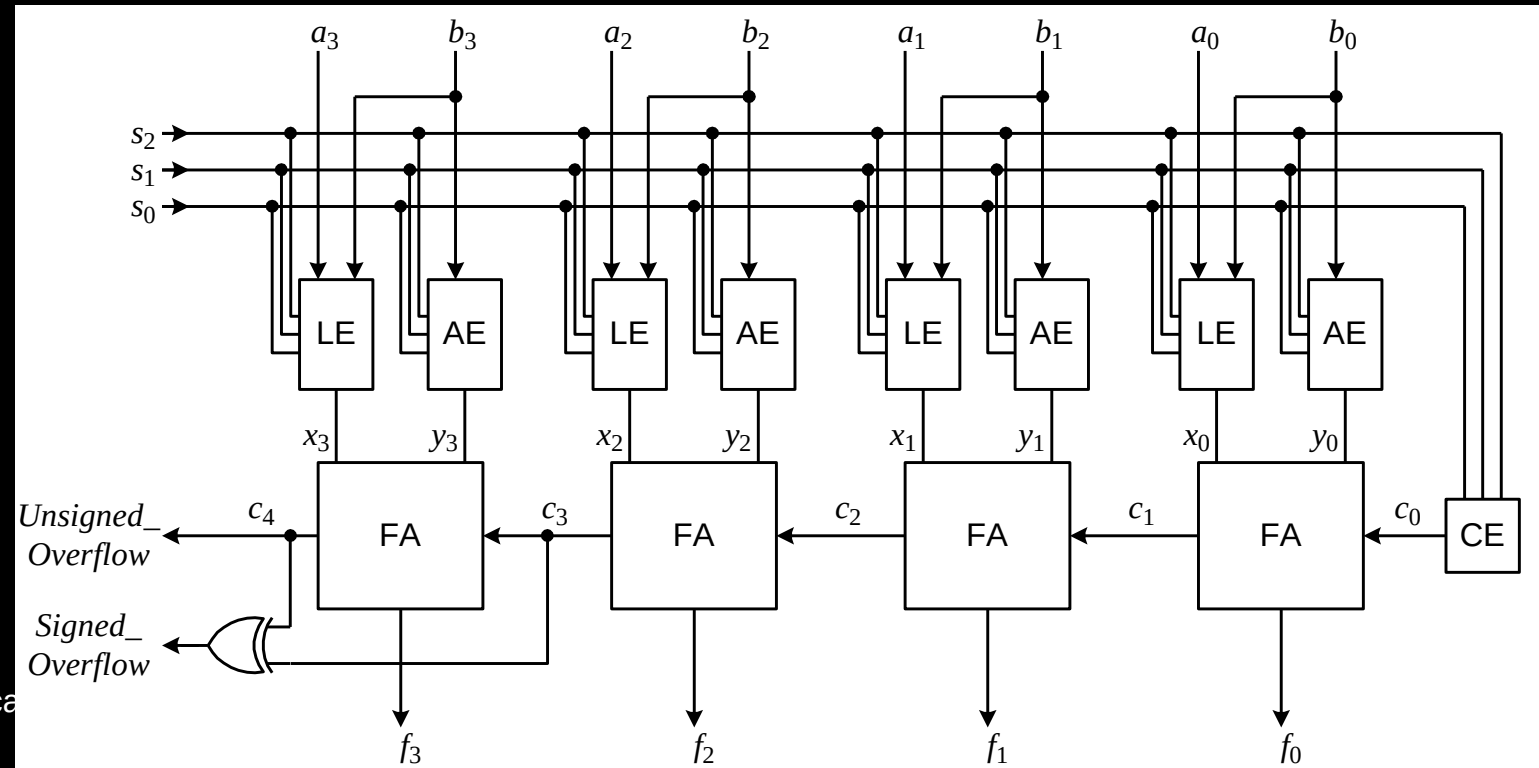
Quiz 4.3

Arithmetic Logic Unit (ALU)

- Arithmetic and logic operations
 - addition, subtraction, logical AND, logical OR, etc.
 - building block: ripple-carry adder & combinational logic circuit

ALU

- 4 Bits ALU Circuit template
 - LE: logic extender
 - AE: arithmetic extender
 - CE: carry extender

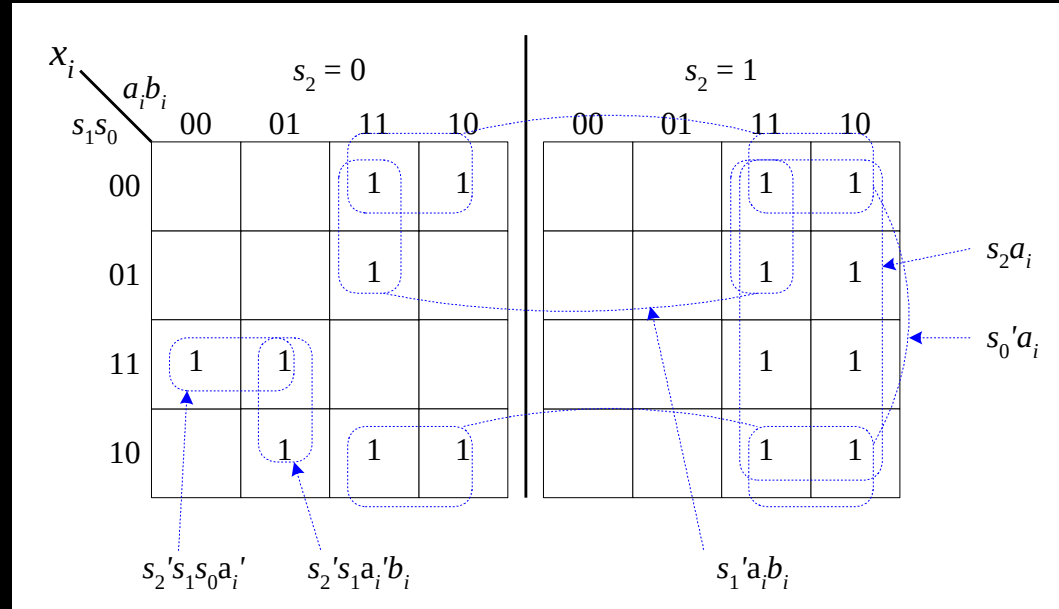


ALU Operation

s_2	s_1	s_0	Operation Name	Operation	x_i (LE)	y_i (AE)	c_0 (CE)
0	0	0	Pass	Pass A to output	a_i	0	0
0	0	1	AND	$A \text{ AND } B$	$a_i \text{ AND } b_i$	0	0
0	1	0	OR	$A \text{ OR } B$	$a_i \text{ OR } b_i$	0	0
0	1	1	NOT	A'	a_i'	0	0
1	0	0	Addition	$A + B$	a_i	b_i	0
1	0	1	Subtraction	$A - B$	a_i	b_i'	1
1	1	0	Increment	$A + 1$	a_i	0	1
1	1	1	Decrement	$A - 1$	a_i	1	0

ALU - LE

s_2	s_1	s_0	x_i
0	0	0	a_i
0	0	1	$a_i b_i$
0	1	0	$a_i + b_i$
0	1	1	a_i'
1	×	×	a_i

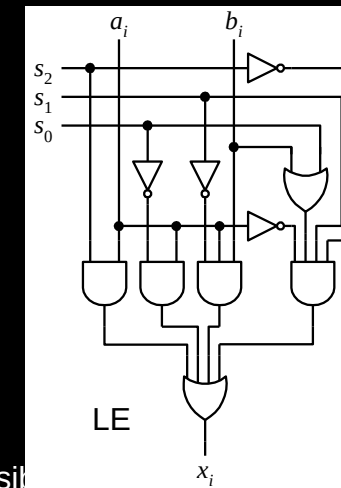


$$x_i = s_2 a_i + s_0' a_i + s_1' a_i b_i + s_2' s_1 a_i' b_i + s_2' s_1 s_0 a_i'$$

$$= s_2 a_i + s_0' a_i + s_1' a_i b_i + s_2' s_1 a_i' (b_i + s_0)$$

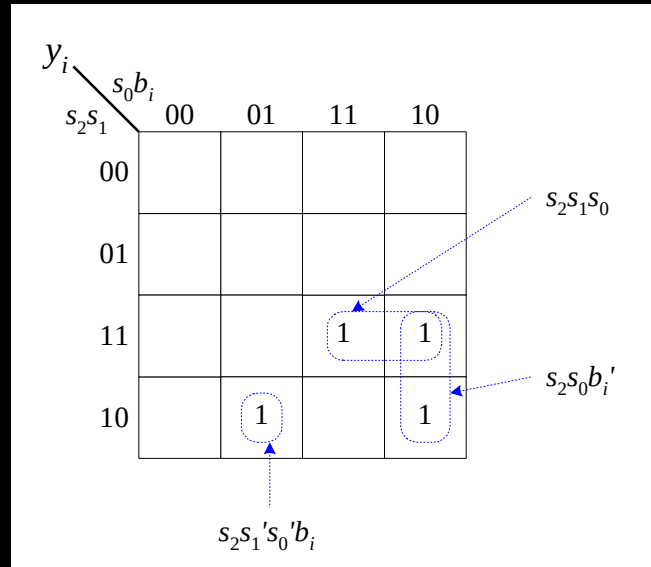
Or

$$x_i = s_2 a_i + s_0' a_i + s_1' a_i b_i + s_2' s_1 s_0' b_i + s_2' s_1 s_0 a_i'$$



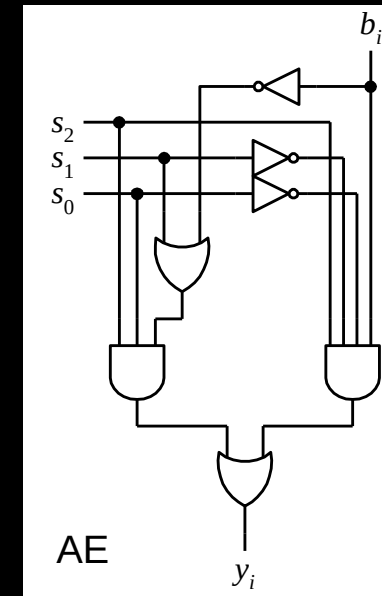
ALU - AE

s_2	s_1	s_0	b_i	y_i
0	×	×	×	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	1
1	1	1	1	1



1	0	0	Addition	$A + B$		b_i	0
1	0	1	Subtraction	$A - B$		b_i'	1
1	1	0	Increment	$A + 1$		0	1
1	1	1	Decrement	$A - 1$		1	0

$$\begin{aligned}
 y_i &= s_2 s_1 s_0 + s_2 s_0 b_i' + s_2 s_1' s_0' b_i \\
 &= s_2 s_0 (s_1 + b_i') + s_2 s_1' s_0' b_i
 \end{aligned}$$



ALU - CE

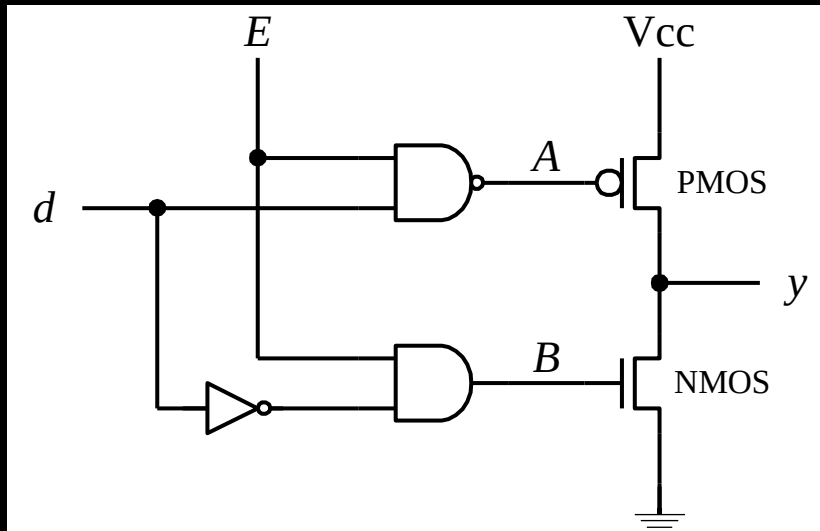
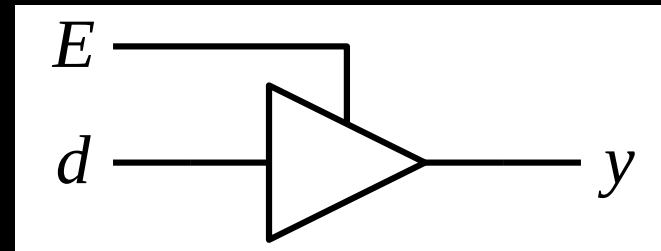
s_2	s_1	s_0	c_0
0	×	×	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

$$c_0 = s_2(s_1 \oplus s_0)$$

Tri-state Buffer

- *three states*
 - 0, 1, and high-impedance state Z

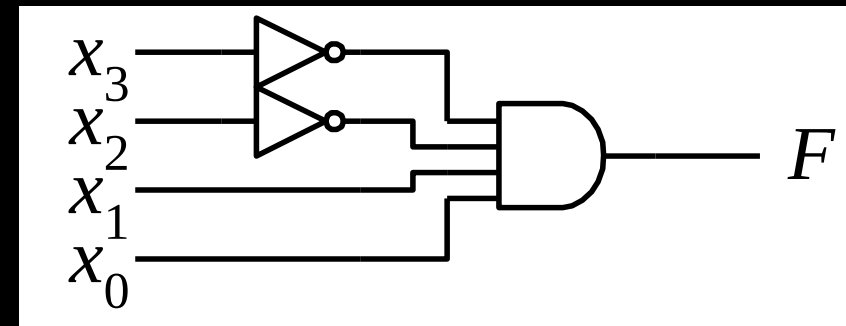
E	y
0	Z
1	d



E	d	A	B	y
0	0	1	0	Z
0	1	1	0	Z
1	0	1	1	0
1	1	0	0	1

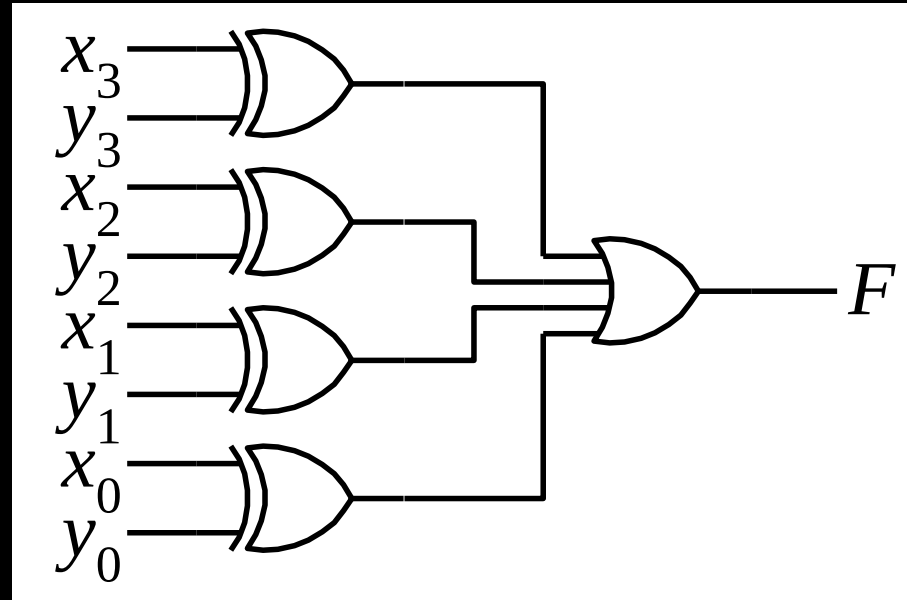
Comparator

- Compares two binary values (equal, greater, less than, etc.)
 - indicates whether the relationship is true (1) or not (0)
- Compare equality with a constant
 - E.g., compare a 4-bit variable x with the constant 3 (0011)



Comparator

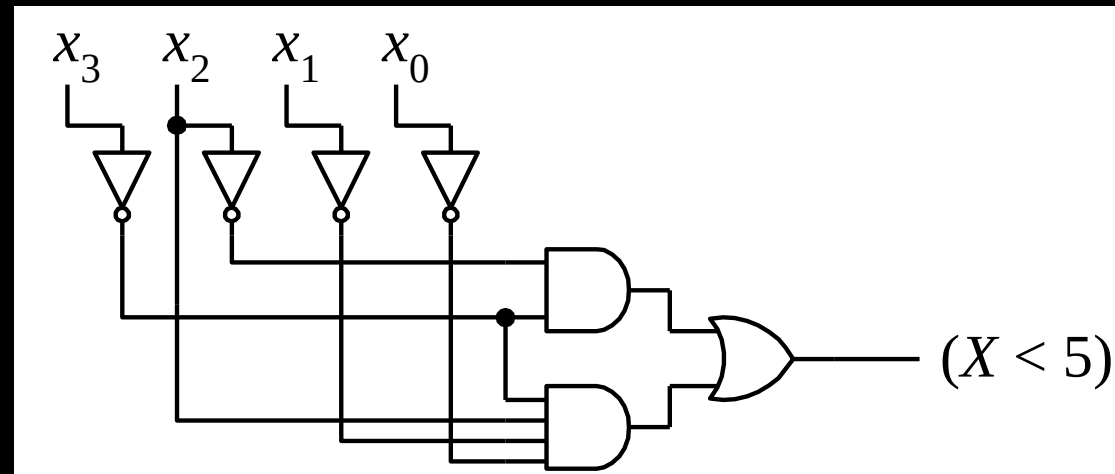
- Compare inequality between two variables
 - E.g., 4-bit comparator $X \neq Y$



Comparator

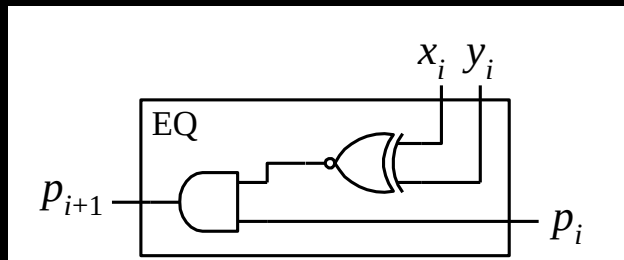
- Compare less than, greater than, etc.
 - E.g., 4-bit comparator $X < 5$

x_3	x_2	x_1	x_0	$(X < 5)$
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	×	×	×	0

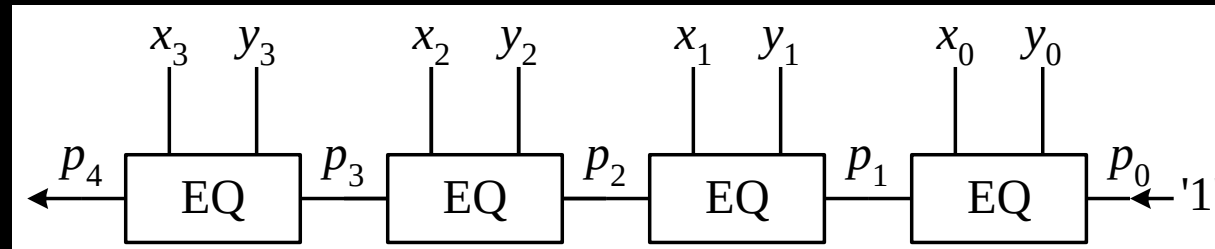


Iterative Comparator

- n-bit comparator
 - *iterative circuit*
 - daisy chaining n of 1-bit slice comparator
 - Input operand x_i and y_i
 - p_i : keeps track of whether all the previous bit pairs compared so far are true or not
 - E.g., (a) 1-bit slice for $x_i = y_i$, (b) 4-bit $X = Y$.



(a)



(b)

Iterative Comparator

- Comparator for $x > y$:
 - status bits, g_{in} and e_{in}
 - when $x_i > y_i$, $g_{in} = 1$; otherwise, $g_{in} = 0$
 - when $x_i = y_i$, $e_{in} = 1$; otherwise, $e_{in} = 0$

g_{in}	e_{in}	x_i	y_i	Meaning	g_{out}	e_{out}
0	0	x	x	<	0	0
0	1	0	0	=	0	1
0	1	0	1	<	0	0
0	1	1	0	>	1	0
0	1	1	1	=	0	1
1	0	x	x	>	1	0
1	1	x	x	Invalid	1	1

$$g_{out} = g_{in} + e_{in}xy'_i$$

$$e_{out} = g_{in}e_{in} + e_{in}x'_iy'_i + e_{in}xy_i$$

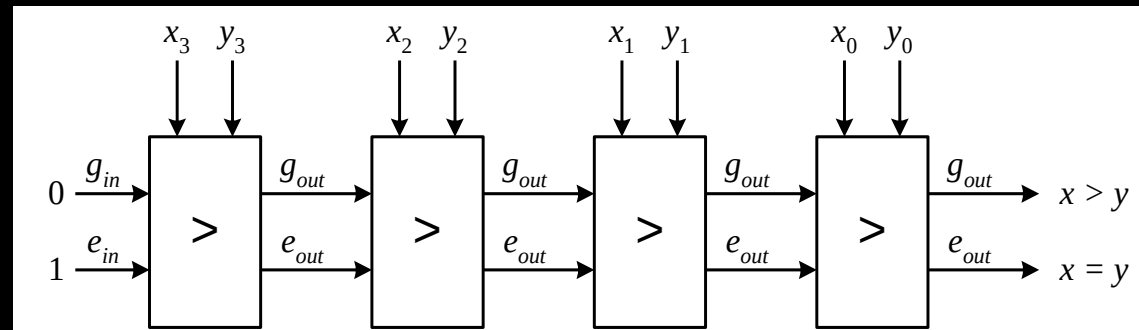
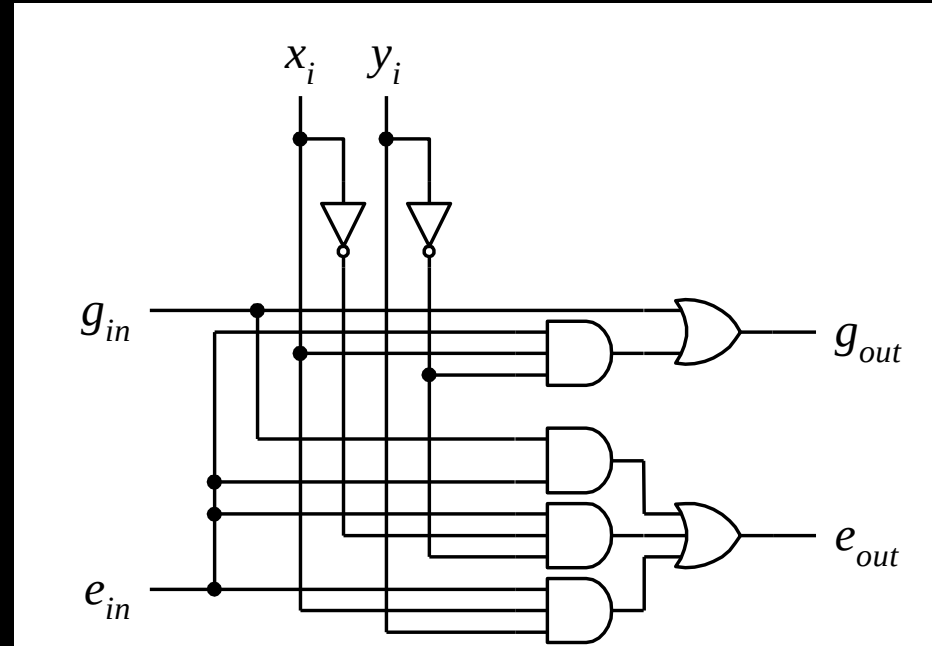
Iterative Comparator

$$g_{out} = g_{in} + e_{in}x_iy_i'$$

$$e_{out} = g_{in}e_{in} + e_{in}x_i'y_i' + e_{in}x_iy_i$$

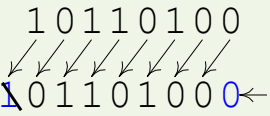
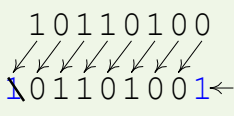
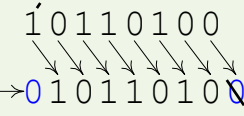
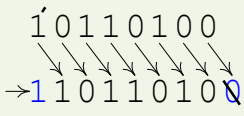
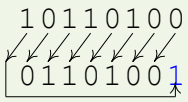
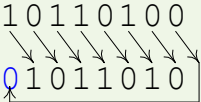
Condition	e_{out}	g_{out}
Invalid	1	1
$x = y$	1	0
$x > y$	0	1
$x < y$	0	0

Operational table



Shifter

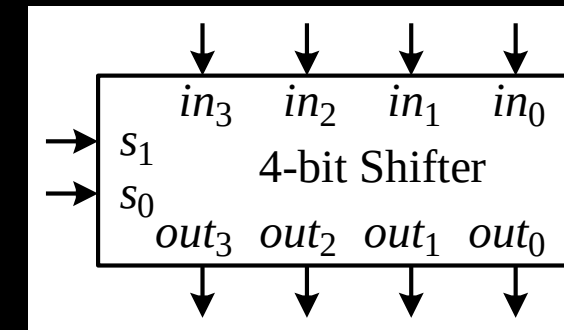
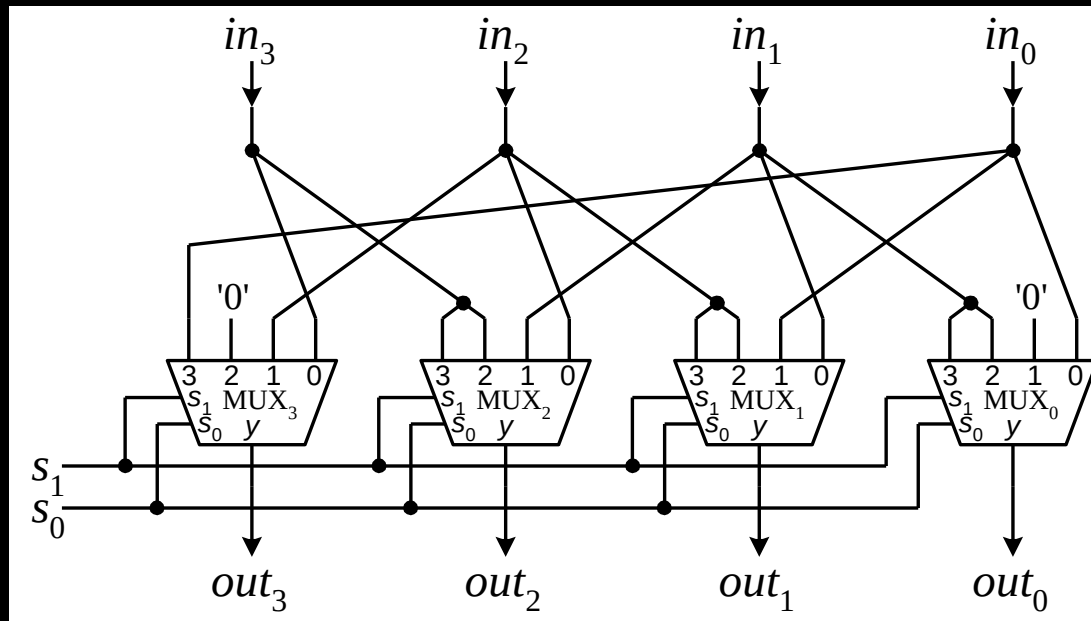
- Shift bits in a binary word one position either to the left or to the right

Operation	Comment	Example
Shift left with 0	Shift bits to the left one position. The rightmost bit is filled with a 0.	
Shift left with 1	the rightmost bit is filled with a 1.	
Shift right with 0	Shift bits to the right one position. the leftmost bit is filled with a 0.	
Shift right with 1	The leftmost bit is filled with a 1.	
Rotate left	The leftmost bit is moved to the rightmost bit position.	
Rotate right	The rightmost bit is moved to the leftmost bit position.	

Shifter

■ A 4-bit shifter

s_1	s_0	Operation
0	0	Pass through
0	1	Shift left and fill with 0
1	0	Shift right and fill with 0
1	1	Rotate right



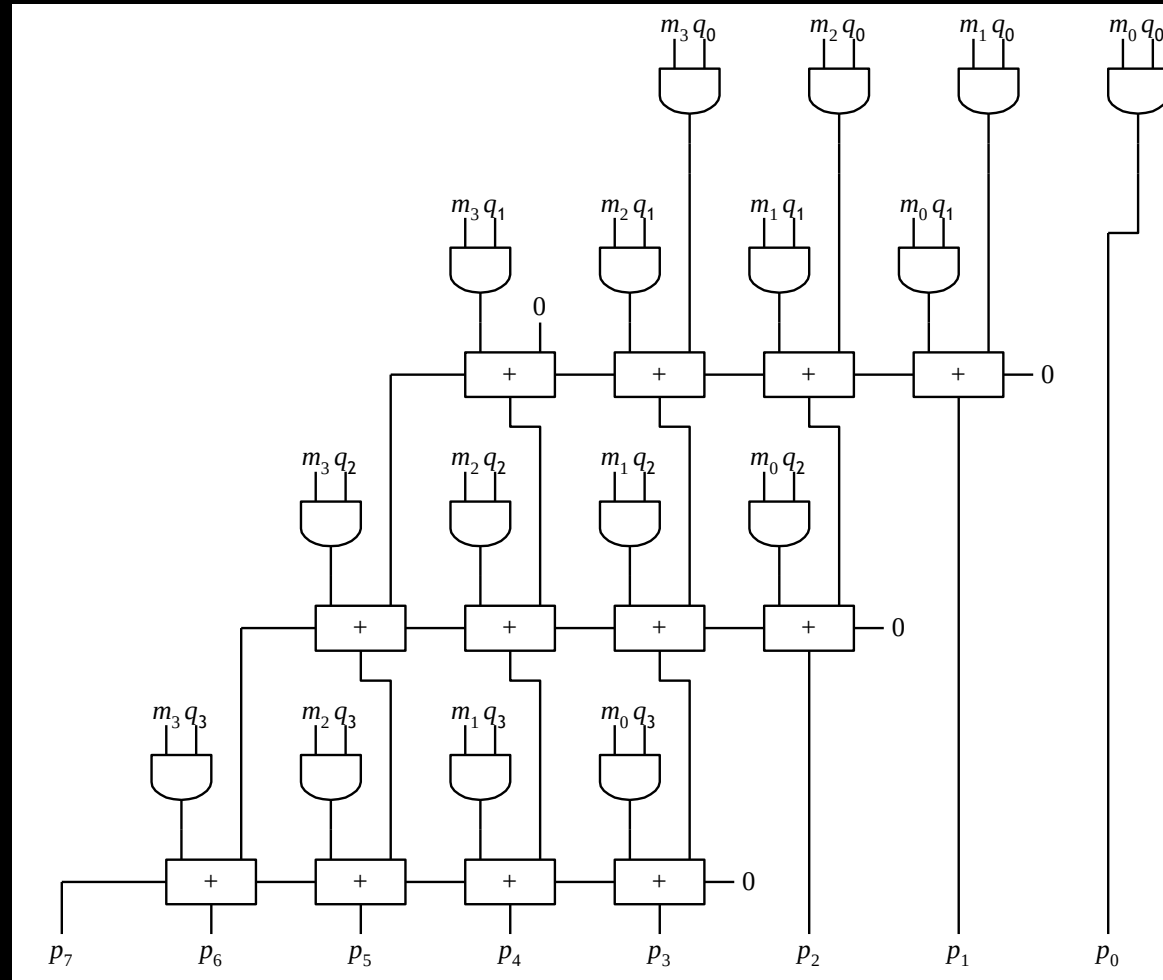
Multiplier

- Multiplication of two 4-bit unsigned binary numbers
 - multiplicand M ($m_3m_2m_1m_0$)
 - multiplier Q ($q_3q_2q_1q_0$)
 - resulting product P ($p_7p_6p_5p_4p_3p_2p_1p_0$)
 - method:

Multiplicand (M)	1 1 0 1	m_3	m_2	m_1	m_0				
Multiplier (Q)	$\times$ 1 0 1 1	$\times$ q_3	q_2	q_1	q_0				
Intermediate products	1 1 0 1	m_3q_0	m_2q_0	m_1q_0	m_0q_0				
	1 1 0 1	m_3q_1	m_2q_1	m_1q_1	m_0q_1				
	0 0 0 0	m_3q_2	m_2q_2	m_1q_2	m_0q_2				
	$+$ 1 1 0 1	$+$ m_3q_3	m_2q_3	m_1q_3	m_0q_3				
Product (P)	1 0 0 0 1 1 1 1	p_7	p_6	p_5	p_4	p_3	p_2	p_1	p_0

Multiplier

- Multiplication of two 4-bit unsigned binary numbers



In this Chapter

- Multiplexer
- Adder
- Subtractor
- Arithmetic Logic Unit
- Decoder
- Tri-state Buffer
- Comparator
- Shifter
- Multiplier

Quiz 4.4